

GAZİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BM496 BİLGİSAYAR PROJESİ II
2023-2024 BAHAR YARIYILI
FİNAL RAPORU

8025 Karakter

Konveyör Bantlarda Görüntü İşleme ile Ürün Takibi ve Paketleme

C181142011 Çağla BAL

C181140068 Seçil SARIGÜL

Prof. Dr. Hacer KARACAN

Tarih: 21.06.2024

ÖZET

Bu proje, görüntü işleme teknikleri ile gerçek zamanlı bir şekilde nesne tespiti yaparak kargo paketlerinin boyutlandırmasını amaçlamaktadır. Günümüz pazarında kargo paketlerinin boyutlandırılması ve tasarlanması büyük bir zorluk oluşturmaktadır. Proje bu zorluğu ele alarak, ürünlerin gerçek zamanlı olarak tespit edilmesini, boyutlarının ölçülmesini ve uygun kargo paketi boyutlandırmasını yaparak, firmalar için kargo paketi tasarımlarının oluşturulmasını otomatize etmek için yenilikçi bir yaklaşım sunmayı hedeflemektedir. Proje çağın en önemli teknolojilerinden olan yapay zekayı kullanmayı amaçlayarak daha önce üzerine çok fazla çalışma yapılmamış bir alan olan boyutlandırma sistemlerini yapay zeka ile birleştirmeyi hedef almaktadır. Bu projede, görüntü işleme teknikleri kullanılarak konveyör banttan geçen nesneleri tespit etmek ve boyutlarını belirlemek amacıyla YOLOv8 ve OpenCV teknolojileri kullanılmıştır. Proje kapsamında, altı farklı sınıfa ait nesneler (kase, sabunluk, krem kutusu, bardak, ruj, kavanoz) için veri seti oluşturulmuş, nesnelerin çeşitli açılardan ve ışık koşullarında çekilen fotoğrafları etiketlenmiştir. Veri seti, toplamda 4000 fotoğraftan oluşmakta olup, eğitim, doğrulama ve test setlerine belirli oranlarda dağıtılmıştır. Eğitim süreci YOLOv8 ile Google Colab kullanılarak gerçekleştirilmiş ve model, 27 epoch boyunca eğitilmiştir. Eğitim sonunda modelin performansı yüksek doğruluk ve genelleme yetenekleri göstermiştir. Modelin eğitimi tamamlandıktan sonra, gerçek zamanlı görüntü işleme algoritmaları kullanılarak nesnelerin boyutları referans obje ile kalibre edilmiştir. Sonuç olarak, proje, nesnelerin doğru bir şekilde tespit edilmesi ve boyutlarının belirlenmesi konusunda başarılı sonuçlar elde etmiştir.

Anahtar Kelimeler: Nesne Tespiti, YOLOv8, OpenCV, Gerçek Zamanlı Görüntü İşleme, Kargo Paketi Boyutlandırma

İÇİNDEKİLER

Sayfa

ÖZET i

İÇİNDEKİLER ii

ŞEKİLLERİN LİSTESİ iv

TABLOLARIN LİSTESİ v

1. GİRİŞ 1

2. TEORİ 2

2.1. Görüntü İşleme 2

2.2. Derin Öğrenme 2

2.3. Derin Öğrenme ile Nesne Tespiti 3

2.3.1. Ön İşleme 3

2.3.2. Nesne Tespiti 4

2.3.3. Nesne Sınıflandırma 4

2.3.4. Nesne Takibi 4

3. LİTERATÜR TARAMASI 5

3.1. Görüntü İşleme Tekniği Kullanılarak Gerçek Zamanlı Hareketli Görüntü Tanıma 5

3.2 Görüntü İşleme Teknikleri ve Kümeleme Yöntemleri Kullanılarak Fındık Meyvesinin Tespit ve Sınıflandırılması 5

3.3. Derin Öğrenme ile Resim ve Videolarda Nesnelerin Tanınması ve Takibi 5

3.4. Yapay Zeka Kullanarak Nesne Tespiti ve Mesafe Ölçümü 6

3.5. Opencv Kullanarak Nesne Algılama ve Boyutlandırma 6

3.6. Gerçek Zamanlı Nesne Algılama ve Yükseklik Ölçümü 6

3.7. Görüntü İşleme Kullanarak Gerçek Zamanlı Nesne Ölçümü 7

3.8. Görüntü İşleme ve Büyük Veri Kullanılarak Kovid-19 Ortamında Sosyal Mesafenin Gerçek Zamanlı Korunması 7

4. PROJENİN GEREKSİNİMLERİ 8

4.1 Ürün Perspektifi 8

4.2 Ürün İşlevleri 8

4.3 Kullanıcı Özellikleri 8

4.4 Kısıtlar 8

4.5. Fonksiyonel Gereksinimler 8

4.5.2. Ürün Boyutlandırma 8

4.5.3 Veritabanına Kayıt 8

4.5.4. Kullanıcı Arayüzü 9

4.5.5. Kullanıcı Yönetimi 9
4.6. Performans Gereksinimleri 9
4.7. Güvenlik Gereksinimleri 9
4.8. Yazılım Kalitesi Gereksinimleri 9
5. PROJENİN TASARIMI 9
6. PROJENİN GERÇEKLEŞTİRİLMESİ 12
6.1. Verisetinin Oluşturulması 12
6.2. Veriseti Eğitimi Ve Model Oluşturma 13
6.2.1. Veriseti Eğitimi Kodları 13
6.2.2. Modelin Performans Metikleri Değerlendirilmesi 16
6.3. Görüntü İşleme Algoritması 19
6.4. Kullanıcı Arayüzü Ve Veritabanı 22
6.5. Konveyor Bant Tasarımı 26
7. SONUÇ VE TARTIŞMA 28
8. PROJE YÖNETİMİ 31
8.1. İş-Zaman Çizelgesi 31
8.2. Risk Yönetimi 32
KAYNAKÇA 33

ŞEKİLLERİN LİSTESİ

Sayfa

Şekil 1. Derin ağ mimarisi 3
Şekil 2. Sınıfların verisetindeki dağılımı 12
Şekil 3. 27 Epoch performans metrikleri grafikleri 17
Şekil 4. Modelin yaptığı tahminler 18
Şekil 5. Görüntü işleme algoritması kod-1 19
Şekil 6. Görüntü işleme algoritması kod-2 20
Şekil 7. Görüntü işleme algoritması kod-3 20
Şekil 8. Görüntü işleme algoritması kod-4 21
Şekil 9. Görüntü işleme algoritması kod-5 21
Şekil 10. Görüntü işleme algoritması kod-6 22
Şekil 11. Arayüz hoşgeldiniz sayfası 23
Şekil 12. Giriş yap sayfası 24
Şekil 13. Kayıt ol sayfası 24
Şekil 14. Kameraların başlatıldığı ve ölçümlerin yazıldığı tablo 25
Şekil 15. Projenin models.py kodları 26
Şekil 16. Django Veri tabanı Arayüzü 26
Şekil 17. Konveyör Bant 27
Şekil 18. Lipstick nesnesinin üst açıdan kamera ile tespiti ve boyutlandırılması 28
Şekil 19. Lipstick nesnesinin yan açıdan kamera ile tespiti ve boyutlandırılması 28
Şekil 20. Cup nesnesinin üst açıdan kamera ile tespiti ve boyutlandırılması 29
Şekil 21. Yan açı genişlik ölçümü 30
Şekil 22. Yan açı uzunluk ölçümü 29
Şekil 23. Cup nesnesinin yan açıdan kamera ile tespiti ve boyutlandırılması 30

TABLOLARIN LİSTESİ

Sayfa

Tablo 1. Epoch sayılarına göre performans metrikleri 18
Tablo 2. Proje Aşamaları 31
Tablo 3. Risk Yönetimi 32

1. GİRİŞ

Günümüzde kargo firmalarında seri üretimde ürün toplama ve paketleme işlemleri genel olarak insanlar ile yapılır. İnsan çalışan sistemlerde verim sürekli yüksek olmamakla birlikte insanın hata yapma olasılığı yüksektir. Bu nedenle seri üretimler için bir paketleme sistemi gereklidir. Bu sistemler ürünlerin kameralar ile algılanması ve nesnelerin tespiti işlemlerini içerebilir. Ayrıca iş gücü kaybını önleyen, hata yapma ihtimalini azaltan ve verimliliği yüksek tutan bu sistemlerin kullanılmasıyla ürünlerin doğru paketlenmesi ile son kullanıcıya hatasıza yakın olarak teslim edilmesi amaçlanmaktadır [1]. Bu amaçlar doğrultusunda sürekli olarak gelişen bilgisayarlı kontrol, otomatik makine sistemleri ve kameralar sayesinde bu işlemler, giderek otomatik bir hale gelmekte ve bilgisayarlı kontrol sistemleri ile hızlı, güvenli ve daha doğru bir şekilde gerçekleştirilmektedir [1]. Bilişim teknolojilerine bakıldığında görüntü işleme, makine öğrenmesi ve yapay zekâ liderlik etmektedir. Akıllı bilgisayarlar, robotlar ve gelişmiş otomasyon sistemleri günümüzde yaygınlaşmaya başladı. Bu sistemlerin en büyük avantajları, uzaktan kontrol ve müdahale yeteneği ve düşük bakım ve kurulum maliyetleridir. Günümüzde bilgisayarlı sistemlerin kullanımının en önemli nedenlerinden biri, teknolojik imkanların sürekli olarak gelişmesidir [2].

Literatürde, renk, desen ve boyutu kontrol eden görüntü işleme tabanlı birçok çalışma bulunmaktadır. Bu çalışmaların amacı, farklı alanlar ve metotlarda yapılan ürün hatalarını tespit etmek ve sınıflandırmaktır. Bu çalışmalarda Yusuf ve ark. görüntü işleme ile gerçek zamanlı ürün takibi sayesinde konveyörden gelen ürünlere kalite kontrol işlemi uygulamış ve çalışma başarıyla sonuçlandırılmıştır. Bir başka çalışmada Yılmaz ve ark. (2023) çikolata üretiminde konveyör banttan gelen çikolataları toplama ve paketleme çalışması yapmıştır. Bu sayede hem zamandan kazanılmış hem daha verimli bir üretim süreci elde edilmiştir. Serdar ve ark. (2017) ise yaptıkları çalışmada görüntü işleme teknikleri ve kümeleme yöntemleri kullanılarak fındık meyvesinin tespiti boyutunun algılanması ve bu şekilde sınıflandırılmasını sağlamışlardır.

Bu çalışmada ise görüntü işleme ve derin öğrenme ile gerçek zamanlı nesne tespiti ve boyutlarının belirlenmesi amaçlanmıştır. Kargo ürünlerinin gerçek zamanlı olarak görüntü işleme ile nesne tespiti ve boyut analizi yapılarak boyutlara uygun biçimde paketleme yapılması istenmiştir. Projenin önemi ve yapılmak istenmesinin nedeni ise endüstride kargolama işlemlerinde ürünlere uygun paketlenme yapılmamasından kaynaklı olarak hem kargolama şirketlerinin yanlış boyutlarda kutulama yaparak zarara girmesidir hem de müşterilerin ürünlerin yanlış paketlenmesi sonucu mağdur olmasıdır. Bu problemleri daha hızlı ve verimli şekilde çözmek için görüntü işleme kullanılarak her ürünün boyutuna uygun paketleme işlemleri yapılacaktır.

Projenin kapsamı, her ürünün boyutunun ve şeklinin doğru bir şekilde tespit edilmesini içerir. Ayrıca, konveyörden gelen hareketli ürünlere boyut analizi yapılacağından işlemlerin hızlı ve kesintisiz bir şekilde gerçek zamanlı yapılmasını gerektirir. Projenin sınırlamaları, görüntü kalitesi, nesne çeşitliliği, donanım sınırlamaları ve entegrasyon zorlukları gibi faktörlerden etkilenmektedir fakat projede kabul edilebilir hata oranlarının korunması önemlidir. Bu hata oranları korunarak proje başarıyla tamamlanmıştır.

Projede konveyör bant tasarımı yapılmış ve hayata geçirilmiştir. Ayrıca uygun veri seti oluşturma eğitme ve görüntü işleme algoritmalarının uygulanması ile ürünlerin tespiti, sınıflandırılması ve iki farklı açıdan kameralar ile boyutlarının belirlenmesi başarılmıştır.

2. TEORİ

BM496 Lisans Bitirme Projesi kapsamında gerçekleştirilecek olan "Görüntü İşleme ve Derin Öğrenme ile Konveyör Bantlarda Nesne Tespiti" projesinde görüntü işleme ve derin öğrenme yöntemleri kullanılarak YoloV8 algoritması ile nesne tespiti yapılmıştır. .Bu bölümde çalışmada kullanılmış ve uygulanmış olan yöntemlerin tanımlarına yer verilmiştir.

2.1. Görüntü İşleme

Görüntü işleme, izleme, segmentasyon, tespit, tanımlama ve sınıflandırma için kullanılan popüler bir tekniktir. Bu teknik, yüzeydeki öğelerin konumu ve boyutuna ilişkin verileri dağınık bir şekilde çok hızlı ve güvenilir bir şekilde toplar. Gelişen teknolojiler arasında, yeni nesil üretim hatlarına rastgele konulan farklı boyutlardaki ürünleri tanımlamak için özel olarak tasarlanmış görüntü işleme yöntemleri yer almaktadır [3]. Modern teknoloji sayesinde herhangi bir görüntü, orijinal görüntüye bağlı gerekli özelliğe veya verilere sahip başka bir görüntü oluşturmak için girdi olarak kullanılabilir. Doğru yazılımla bir resmin rengi, parlaklığı, boyutu ve yapısı gibi unsurlar görüntü işleme süreci aracılığıyla değiştirilebilir, iyileştirilebilir ve incelenebilir. Bu programlar, nesne tanımlama ve hareketli ve sabit nesneler arasındaki ayrımın yanı sıra dijital ortama dönüştürülen fotoğraflardaki bozulmaları gidermek ve daha yüksek kalitede fotoğraflar üretmek gibi çeşitli görevler için kullanılabilir. Görüntü işleme, görüntüleri çeşitli formlarda kullanan her işletmeye uygun çözümler oluşturmayı mümkün kılar. Kalite kontrolden astronomiye, güvenlikten askeri sanayiye kadar görüntü işlemeye yönelik sonsuz uygulama bulunmaktadır [4].

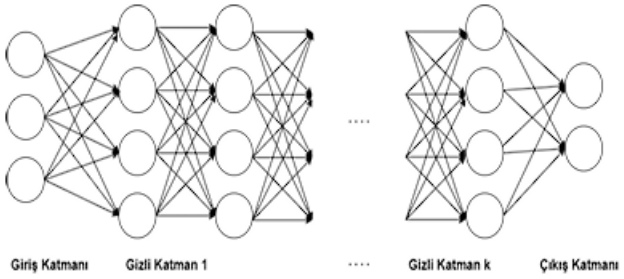
2.2. Derin Öğrenme

Makine öğreniminin bir alt alanına derin öğrenme denir [5]. Ayrıca derin öğrenme insan hatasını azalttığı için son zamanlarda popülerlik kazanan yapay zeka tekniklerinden biridir. Çeşitli sektörlerden gelen büyük hacimli veriler, derin öğrenme algoritmaları kullanılarak etkili bir şekilde tespit edilip yorumlanabilir. Etiketli verilerdeki, özellikle görüntü işleme alanındaki hızlı büyüme nedeniyle artık derin öğrenme yöntemlerine ihtiyaç duyulmaktadır. Derin öğrenme teknikleri, çeşitli alanlardaki büyük miktarda veriden anlamlı bilgiler çıkarmak ve metin, resim ve ses dosyalarını yorumlamak için kullanılır [6].

Yapay sinir ağları (YSA) üzerine yapılan çalışmalar genellikle derin öğrenme teknikleri oluşturmak için kullanılır. Bu bulguların aksine, daha fazla sayıda gömülü nöron ve katmana dayanmaktadırlar. Mevcut verileri çeşitli soyutlama seviyelerinde temsil etmek için derin öğrenme, çeşitli işlem katmanlarına sahip bilgisayar modellerini kullanır. Şekil 1'de gösterildiği gibi, bazen derin ağlar olarak da adlandırılan derin öğrenme tekniklerinde verileri temsil etmek için birçok katman üst üste dizilir [5].

Derin öğrenme yöntemlerinin daha ayrıntılı olarak verilebilecek uygulama alanları arasında aşağıdakiler yer almaktadır:

1. Dil modelleme ve doğal dil işleme (language modeling and natural language processing)
2. Konuşma ve ses işleme (speech and audio processing)
3. Bilgi erişimi (information retrieval)
4. Nesne tanıma ve bilgisayarlı görü (object recognition and computer vision)
5. Çok modlu ve çok görevli öğrenme (multimodal and multitask learning) [7]



Şekil 1. Derin ağ mimarisi [3]

2.3. Derin Öğrenme ile Nesne Tespiti

Son zamanlarda nesne takibi ve tespiti alanında yapılan çalışmaların sayısı arttı. Videolar gibi statik olmayan resimler üzerinde tanımlama ve analiz sonrasında bir öğenin takip edilmesi gerektiğinde ilgili bilgilerin alınması daha zordur. Bu durumlarda görüntü işleme sorunlarının çözülmesini kolaylaştırmak için derin öğrenme yöntemleri kullanılır.

Görüntüler, görüntü işleme teknikleri kullanılarak elde edilebilecek değerli bilgiler içerebilir. Filmlerde tespit edilip analiz edildikten sonra takip edilmesi gereken bir öğe varsa ilgili bilgiyi çıkarmak daha zordur. Bu durumlarda görüntü işleme sorunlarının çözülmesini kolaylaştırmak için derin öğrenme yöntemleri kullanılır. Fotoğraflardaki öğeleri hızlı bir şekilde tanımlayan ilk algoritmaya Viola Jones adı verildi.

Derin öğrenme ile nesne tespiti ve takibi alanında yapılan çalışmalar incelendiğinde genel olarak dört adım uygulanmaktadır. Bunlar;

• Ön işleme aşaması,

• Nesnenin tespiti,

• Nesnenin sınıflandırılması

• Nesnenin takibidir [7].

2.3.1. Ön İşleme

Ön işleme, verileri işlemeye hazırlamak için önceden belirlenmiş kategorilere göre oluşturulan veri setleri üzerinde yapılan ilk işlemlerdir. Ön işleme, sistemik hataları telafi etmeye veya düzeltmeye çalışan ve verileri daha sonraki analizler için hazırlayan bir dizi prosedürdür. Dijital fotoğraflar atmosferik ve geometrik olanlar da dahil olmak üzere çeşitli değişikliklere uğrar. Bazı durumlar bu tedavilerin tümünü gerektirmeyebilir. Bu hatalar kullanıma sunulmadan önce sistematik olarak düzeltilir [7].

2.3.2. Nesne Tespiti

Videolarda tanımlanan nesne algılama aşamasında iki süreç bulunmaktadır. Amaç ilk aşamada ortaya çıkar. İkinci aşamada öğe arka plandan çıkarılır. Şekil ve görünümle ilgili özellikler sıklıkla nesne tanımanın temelini oluşturur. Bir öğe için uygun izleme algoritmasını seçen adıma nesne algılama adı verilir. Nesne algılama adımı, diğer aşamaların ne kadar iyi çalıştığını etkilediği için daha önemlidir. Nesnenin tespiti ve takibinde kullanılan metotlar;

- R-CNN (Region Based Convolutional Networks),
- YOLO (You Only Look Once),
- SSD (Single Shot Multibox Detector),
- Mask R-CNN (Ozbaysar ve Borandag, 2018),
- Fast R-CNN,
- Faster R-CNN'dir [5].

YOLO, doğrudan algılama performansına dayalı olarak nesneleri tespit etmek amacıyla sorunu baştan sona optimize etmek için tek bir sinir ağı kullanır. Özellikle son birkaç yıldır en yaygın kullanılan nesne tanımlama yöntemlerinden biridir. Nesne tespitinde önemli bir fikir hızdır. YOLO'nun gerçek zamanlı nesne tanımlaması onu mevcut tekniklerden ayırır [5].

2.3.3. Nesne Sınıflandırma

Sınıflandırma, öğeleri özelliklerine göre düzenleme işlemidir. Kategorizasyon prosedürü, nesnenin ayırt edici özelliklerinden yararlanır. Nesne sınıflandırmasında en sık kullanılan özellikler doku, eğim, zaman, renk ve harekettir. Renk bazlı nesne sınıflandırması iyi çalışıyor ancak ışığa duyarlı olduğu için hesaplanması uzun zaman alıyor. Hesaplama süresi ve doğruluğu yüksek olan bir özellik türü dokudur. İnsan takibi için eğim ve hareket öznitelik türleri kullanılabilir. Sese duyarlıdır. Hareket ve zamansal değişiklikler için uzun hesaplama süreleri nedeniyle maliyet önemlidir [5].

2.3.4. Nesne Takibi

Nesneleri takip etmek için çeşitli algoritmalar kullanılır. Bazı algoritmalar birçok nesneyi izlemek için kullanılırken bazıları yalnızca birini izler. Nesne izleme algoritmaları kategorize edilebilir veya büyük gruplar halinde gruplandırılabilir. 3 kategori yaygın olarak kullanılan nesne izleme tekniklerini içerir. Bunlar;

- Nokta Tabanlı
- Çekirdek Tabanlı
- Siluet Tabanlı'dır [5].

3. LİTERATÜR TARAMASI

3.1. Görüntü İşleme Tekniği Kullanılarak Gerçek Zamanlı Hareketli Görüntü Tanıma

Bu çalışmada, kamera kullanılarak cihazdan alınan ardışık görüntülerde hareketin analizi için kullanılan yöntemler incelenip uygulanmıştır. Bu yöntemler arka plan farkı yöntemleri ve istatiksel yöntemler olmak üzere iki kategori de incelenmiştir. Yöntem olarak hareket tespiti segmentasyon algoritmalarından olan arka plan farkı yöntemlerinden basit fark alma yöntemi uygulanmıştır [4]. Bu yöntemde, hareketli bölgeler, incelenen görüntü ile referans görüntünün aynı, koordinatlardaki piksel farklarının, önceden belirlenmiş bir eşik değeriyle kıyaslanması sonucu tespit edilir. Bir sonraki aşamada yüz tanıma çalışmalarına temel teşkil eden yüz belirleme işlemi gerçek zamanlı olarak gerçeklenmiştir. Yüz bulma işlemi için ten rengi algoritmalarından yararlanılmıştır. Son aşamada web kamerasının hareketi takip etmesini sağlamak için bir mekanik sistem gerçekleştirilmiştir. Adım Motoru kullanılarak hazırlanan mekanizma nesnenin hareketli piksellerinin ağırlık merkezine göre dönerek hareketli nesneyi sürekli olarak takip etmektedir. Bu şekilde kişisel bilgisayarın (PC) paralel portuna bağlı adım motoru düzeneğinin üzerinde bulunan web kamerası, sürekli olarak hareket eden nesne/nesneleri takip edecek şekilde yönlendirilmektedir [4].

3.2 Görüntü İşleme Teknikleri ve Kümeleme Yöntemleri Kullanılarak Fındık Meyvesinin Tespit ve Sınıflandırılması

Bu çalışma, gerçek zamanlı fındık meyvesi görüntülerinin tespit ve sınıflandırılması amacıyla görüntü işleme tekniklerini kullanır. Fındık meyvesi, K1 (küçük), K2 (orta) ve K3 (büyük) olmak üzere üç sınıfa ayrılmıştır [8]. Ortalama tabanlı sınıflandırma ve K-means kümeleme yöntemleri kullanılarak bu işlem gerçekleştirilmiştir. Veriler, bilgi veri tabanından alınmış ve K-means kümeleme yöntemi ile sınıflandırılmıştır. Çalışmada 1.3 Megapiksel CMOS, 640 x 480 çözünürlüğe sahip Logitech C110 USB kamera kullanılmıştır. Alınan görüntüler Ubuntu 12.04 işletim sistemine sahip bir bilgisayar üzerinde işlenmiştir [8]. Görüntülerin işlenmesi ve sınıflandırılması aşamalarında OpenCV Kütüphanesi ve Weka yazılımı kullanılmıştır. Görüntü ön işleme aşamasında, resimlere filtreleme, grileştirme, eşikleme ve morfolojik işlemler uygulanmıştır. Bu işlem sonucunda ilgili nesnelerin dış hatları belirlenmiştir. Çalışmada kullanılacak alan, çap, yarıçap ve merkez noktasına ait koordinatlar belirlenmiştir [8].

3.3. Derin Öğrenme ile Resim ve Videolarda Nesnelerin Tanınması ve Takibi

Bu çalışmada derin öğrenme ile hareketli nesne tanıma ve takibi için Google'ın açık kaynak kodlu makine öğrenmesi kütüphanesi olan TensorFlow kullanılmıştır. Nesne takibi için Region Based Convolutional Networks kütüphanesinden Faster R-CNNmodeli ele alınmıştır [9]. Bu kütüphaneler ile durağan görüntüler, video görüntüleri ve webcam görüntüleri üzerinde nesne tanıma işlemi gerçekleştirilmiş ve incelenen kütüphanelerin güçlü ve zayıf yönleri ortaya konmuştur. Sonuç olarak bu çalışmada Faster R-CNN modeliyle resim, video, webcam verileri üzerinde kullanılarak nesne tanıma işlemi yapılmıştır. Öncelikle object detection hazır eğitilmiş kütüphanesi kullanılarak video, resim ve webcam verileri için nesne algılama yapılmıştır. Resimler tek tek LabelImg ile etiketlenip xml dosyaları oluşturulmuştur. Oluşturulan sınıf kadar her resime id atanmıştır. Bu ID'lere sınıf isimleri atanmıştır. Yeterli düzeyde eğitim gerçekleştikten sonra geliştirilen model iskambil kartlarını tanımayı öğrenmiştir [9].

3.4. Yapay Zeka Kullanarak Nesne Tespiti ve Mesafe Ölçümü

Mustafa F. Ve arkadaşları bu araştırma çalışmalarında, sürücülerin güvenli yollar sağlamasına ve bakım kazalarını azaltmasına veya ortadan kaldırmasına yardımcı olmayı amaçlamaktadır. Nesne tespiti, görüntülerdeki nesneleri bulmak ve konumlandırmak için kullanılan bir tekniktir. Bu çalışmalarda YOLO Versiyon 3, hızı, basitliği, nesneleri tahmin etme ve sınıflandırma yeteneği nedeniyle çerçevedeki nesneyi algılamak için kullanılan ağdır. Ayrıca kabinin yönünü ölçmek için direksiyon açısı devresi tasarlanıp uygulanmıştır. Direksiyon açısı ölçümleri, nesne algılamayla (araçlar ve yayalar) bu nesneler sürüş yapan araca yakın olduğunda (10 metre) uyarı vermek için kullanılır. YOLO V3 kullanılarak nesneler tespit edildikten sonra, nesnenin yüksekliği kullanılarak algılanan nesnelerin mesafesi ölçülür. Mesafe değerlerini tespit etmek için ikinci ağ kullanılmaktadır. Bu ağ, YOLO'nun sınırlayıcı kutusunu ve kameranın odak uzaklığını kullanarak nesnenin otonom arabaya olan mesafesini tahmin etmek için kullanılmasını öngörmüşlerdir. Son adım direksiyon açısına bağlı olarak uyarı alanını çizmektir. Bu, arabanın ön paneline monte edilmiş bir web kamerasından gelen bir girdi kullanılarak elde edilmiştir. Güvenlik sistemi, otomobilin her iki yanına daha fazla kamera eklenerek geliştirilebilir ve otomobilin her tarafının görülebilmesini sağlamıştır [10].

3.5. Opencv Kullanarak Nesne Algılama ve Boyutlandırma

Dr M.Mahesh ve arkadaşları bu çalışmada, web kamerası gibi bir IoT video cihazından nesneleri tespit etmek ve gerçek zamanlı ölçümlerini hesaplamak için artırılmış bir teknik sunmaktadır. Sırasıyla OpenCV kütüphaneleri ve web kamerası gibi AI ve IoT teknolojilerini kullanarak gerçek zamanlı bir nesne ölçüm tekniği önermişlerdir. OpenCV, kullanılan birçok kütüphane ve algoritmayı içerir. Ve bu yazıda akıllı kenar algılama algoritması ve mobil/web kamera yakalama videosu gibi Algoritmalar kullanmaktadırlar.

Sonuç olarak, yönleri aşamalı olarak ölçmek için PC görüşü (web kamerası cihazı ve kodu) kullanmışlardır. Resmi gerçek zamanlı video çerçevesinden yakalar ve ardından yönlerini göstermişlerdir. Canny kenar bulucu, yönleri tanımlamak için etkili bir şekilde kullanılmıştır. Bu prosedür hızlı çalışır ve gerçekte gerçekleştirilebilecek birçok avantaja ve dikkate değer özelliklere sahiptir [11].

3.6. Gerçek Zamanlı Nesne Algılama ve Yükseklik Ölçümü

Neha M. Ve arkadaşları bu yazıda nesneyi tespit etmek ve yükseklik ölçümü yapmak için bir yöntem önermişlerdir. Bu nesne tespit ve yükseklik ölçüm algoritması sayesinde şüpheli kolaylıkla takip edilebilmekte ve şüphelinin boyu ölçülebilmektedir. Bu gerçek zamanlı tespit ve ölçüm algoritması yalnızca şüphelilerden video güvenlik kamerasından şüphelenmek için değil, aynı zamanda birçok çevrimiçi alışveriş platformunda da kullanılmayı düşündüler. Boy ölçümü sayesinde boylarının ölçümünü gösterecektir. Burada çeşitli gerçek zamanlı nesne algılama ve yükseklik ölçümü algoritmalarını ele almışlardır. Bu tezde, vücut boyunu ölçmenin bir yolunu geliştirdiler; bu yöntemle insanlar, vücudun boyunu ve genişliğini ölçerek uygun kıyafetleri bulabilirler.

Görüntüde veya gerçek zamanlı olarak yakalanan nesnenin adını sağlamayı başarmışlardır. Bu sayede şüpheliyi izlemek ve şüphelinin boyunu ölçmek kolaylaşır.

Bu araştırma makalesinde, makine öğrenimi tabanlı gerçek zamanlı nesne algılama ve yükseklik ölçümü için çeşitli koşulları ve teknolojileri ele almışlardır. Burada çeşitli nesne algılama yöntemleri ve yükseklik ölçüm algoritmaları kullanıyoruz. OpenCV'yi kullanarak, hayali ızgara hücreli bağlama kutuları oluşturarak nesneyi tespit edebilir ve gerçek zamanlı olarak merkez referans noktası yardımıyla yüksekliği ölçebildiğini göstermişlerdir [12].

3.7. Görüntü İşleme Kullanarak Gerçek Zamanlı Nesne Ölçümü

Sukesh J. ve arkadaşları bu makalede kamera gibi bir IoT video cihazının nesneleri tanımak ve gerçek zamanlı ölçümlerini hesaplamak için nasıl kullanılabileceğini göstermişlerdir. NumPy, OpenCV kütüphaneleri ve kameralara dayanarak nesnelerin ölçümlerini ve boyutlarını hesaplamak için IoT tabanlı bir gerçek zamanlı nesne ölçüm tekniği önerdiler. Bu projede OpenCV'nin birçok modülü ve algoritması kullanıldı. Bu tekniğin beş aşaması olduğunu belirtmişlerdir; bunlar aşağıdaki gibidir. Resim çekme, Bir nesnenin ölçülmesi, Alanın hesaplanması, Çıktının kaydedilmesi, Ölçümlerin görüntülenmesi. Canlı bir video çerçevesinden alınan görüntüdeki nesnelerin oranlarını göstermiştir. Canny kenar dedektörü kullanılarak boyutlar uygun şekilde tanımlamıştır [13].

3.8. Görüntü İşleme ve Büyük Veri Kullanılarak Kovid-19 Ortamında Sosyal Mesafenin Gerçek Zamanlı Korunması

Bu çalışmada önerilen büyük veri mimarisi ile covid virüsüne önlem amaçlı olan sosyal mesafenin korunması amaçlanmaktadır. Bu amaçla, video akışlarındaki nesnelerin gerçek zamanlı olarak tespit edilmesi ve mesafelerini hesaplamak için gelişmiş bir teknik kullanılmıştır. Sistem, kameradan video akışı, mesafe ve nesne algılama modeli, gelen veri akışı, veri akışı toplama ve rapor oluşturma gibi bir akışın gerçekleştirilmesine yönelik araştırma ve geliştirmelerden oluşmuştur [14].

Kameradan gelen video akışı GStream ile işlenir. Video akışından alınan kareler OpenCV tarafından alınmış, YOLOV3 mesafe ve nesne algılama modeliyle eğitilmiş ve Python tarafından geliştirilmiştir [14]. Önerilen gerçek zamanlı büyük veri mimarisi, video akışı verilerinin Kubeflow ile eğitilmesi ve Apache Kafka ile yayınlanmasını içerir. Bu veriler Apache Spark ile işlenir ve HDFS'de depolanır. Hive, Impala ve Hbase, HDFS'deki verilere sorgular yapmak ve analizler yapmak için kullanılır [14]. Ardından, Apache Oozie kullanılarak veri tabanındaki verilere dayalı e-posta bildirimleri oluşturulabilir. Bu bütünsel yaklaşım, kapalı alanlardaki insanların güvende olmalarını sağlamak için etkili bir veri işleme ve analiz altyapısı sunar. Sonuç olarak bu sistem ile tespit edilen kişiler, mesafe bilgileri, koordinatlar yayına gönderilir. Akıştan veri tabanına gelen veriler, tarihsel ve gerçek zamanlı olarak analitik raporlarda gösterilir.

4. PROJENİN GEREKSİNİMLERİ

4.1 Ürün Perspektifi

Bu yazılım, konveyör bant üzerinde geçen ürünlerin boyutlandırılması ve tespit edilmesi için iki kamerayı kullanacaktır. Sistem, tespit edilen ürünlerin bilgilerini gerçek zamanlı olarak kullanıcı arayüzüne yazacak ve aynı zamanda veritabanına kaydedecektir.

4.2 Ürün İşlevleri

Gerçek Zamanlı Ürün Tespiti: Kameralar aracılığıyla ürünlerin tespiti.

Ürün Boyutlandırma: Tespit edilen ürünlerin boyutlarının hesaplanması.

Veritabanına Kayıt: Tespit edilen ürünlerin veritabanına kaydedilmesi.

Kullanıcı Arayüzü: Sonuçların kullanıcı dostu bir arayüzde gösterilmesi.

Kullanıcı Yönetimi: Kullanıcı kaydı ve giriş işlemlerinin yapılabilmesi.

4.3 Kullanıcı Özellikleri

Operatörler: Sistemi izleyip, tespit edilen ürün bilgilerini kontrol edecek kullanıcılar.

Yöneticiler: Kullanıcı yönetimi ve sistem bakımını yapacak kişiler.

4.4 Kısıtlar

Sistem, yalnızca iki kamera ile çalışacaktır.

Kullanılan veritabanı Django'nun içinde gelen SQLite olacaktır.

4.5. Fonksiyonel Gereksinimler

4.5.1. Gerçek Zamanlı Ürün Tespiti

[FR-1] Sistem, konveyör bant üzerindeki ürünleri iki farklı kameradan tespit etmelidir.

[FR-2] Tespit edilen ürünlerin koordinatları ve boyutları hesaplanmalıdır.

4.5.2. Ürün Boyutlandırma

[FR-3] Her iki kameradan gelen görüntüler işlenmeli ve ürünlerin boyutları hesaplanmalıdır.

[FR-4] Boyutlar santimetre cinsinden hesaplanıp, görüntü üzerine yazdırılmalıdır.

4.5.3 Veritabanına Kayıt

[FR-5] Tespit edilen her ürünün bilgileri SQLite veritabanına kaydedilmelidir.

[FR-6] Veritabanı, ürün kimliği, ürün adı, boyutlar ve tespit zamanı gibi bilgileri içermelidir.

4.5.4. Kullanıcı Arayüzü

[FR-7] Tespit edilen ürünlerin bilgileri bir tabloda gösterilmelidir.

[FR-8] Arayüz, kullanıcı dostu ve operatörlerin kolayca kullanabileceği şekilde tasarlanmalıdır.

4.5.5. Kullanıcı Yönetimi

[FR-9] Kullanıcılar, sisteme kayıt olabilmeli ve giriş yapabilmelidir.

[FR-10] Yönetici paneli, kullanıcı yönetimi ve sistem bakım işlemleri için erişilebilir olmalıdır.

4.6. Performans Gereksinimleri

[PR-1] Sistem, tespit işlemlerini gerçek zamanlı olarak gerçekleştirebilmelidir.

[PR-2] Arayüzdeki bilgilerin güncellenme hızı, kullanıcının algılayamayacağı kadar hızlı olmalıdır (en fazla 1 saniye gecikme).

4.7. Güvenlik Gereksinimleri

[SR-1] Kullanıcı verileri güvenli bir şekilde saklanmalıdır.

[SR-2] Sistem, CSRF koruması içermelidir.

[SR-3] Sadece yetkili kullanıcılar yönetici paneline erişebilmelidir.

4.8. Yazılım Kalitesi Gereksinimleri

[QR-1] Kod, Django, OpenCV ve YOLO kütüphanelerinin en iyi uygulamalarına uygun olmalıdır.

[QR-2] Sistem, hata durumlarında anlamlı hata mesajları vermelidir.

[QR-3] Kullanıcı arayüzü, farklı cihaz ve tarayıcılarda tutarlı bir şekilde çalışmalıdır.

5. PROJENİN TASARIMI

5.1. Sistem Mimari Tasarımı

5.1.1. Genel Sistem Mimarisi

Sistem, üç ana bileşenden oluşmaktadır: Kamera Modülü, Veritabanı Modülü ve Kullanıcı Arayüzü Modülü. Kamera Modülü, konveyör bant üzerindeki ürünleri tespit etmek ve boyutlandırmak için kullanılır. Veritabanı Modülü, tespit edilen ürünlerin bilgilerini saklar. Kullanıcı Arayüzü Modülü ise sonuçları kullanıcıya gösterir ve kullanıcı yönetimi işlemlerini gerçekleştirir. Bu bileşenler arasındaki etkileşim, sistemin genel işleyişini sağlar.

Kamera Modülü

Kamera Modülü, iki ayrı kameradan (Kamera 1 ve Kamera 2) oluşur ve bu kameralar konveyör bant üzerindeki ürünleri farklı açılardan görüntüler. Her iki kameradan alınan görüntüler, YOLO modeli ile işlenerek ürünlerin tespiti yapılır. Tespit edilen ürünlerin boyutları piksel cinsinden hesaplanır ve santimetreye dönüştürülür. Bu işlemler OpenCV kütüphanesi kullanılarak gerçekleştirilir.

Veritabanı Modülü

Veritabanı Modülü, Django'nun varsayılan SQLite veritabanını kullanarak ürün bilgilerini saklar. Bu modül, tespit edilen ürünlerin tüm detaylarını içeren bir Measurement modeli kullanır. Measurement modeli, ürün ID'si, ürün adı, kameralar tarafından ölçülen genişlik ve yükseklik bilgilerini ve zaman damgasını içerir.

Kullanıcı Arayüzü Modülü

Kullanıcı Arayüzü Modülü, Django templateları kullanılarak oluşturulmuş HTML/CSS sayfalarından oluşur. Bu sayfalar, kullanıcı dostu arayüzler sunar ve kullanıcıların sistemle etkileşimini sağlar. Arayüz, tespit edilen ürünlerin bilgilerini dinamik olarak gösteren tablolar içerir. Ayrıca, kullanıcı kayıt ve giriş işlemleri için formlar ve doğrulama mekanizmaları da bu modül tarafından sağlanır.

5.2. Veri Akışı

Kamera Modülü, görüntüleri YOLO modeline gönderir. YOLO modeli, ürün tespitini yapar ve boyut hesaplama modülü, tespit edilen ürünlerin boyutlarını hesaplar. Hesaplanan boyutlar, veritabanına kaydedilir. Kullanıcı Arayüzü Modülü, veritabanından alınan verileri kullanıcıya gösterir. Bu süreç, kullanıcı kayıt ve giriş işlemleri ile desteklenir.

5.3. Modül Tasarımları

5.3.1. Kamera Modülü

Kamera Modülü, iki ana bileşenden oluşur: Kamera 1 ve Kamera 2. Bu kameralar, konveyör bant üzerindeki ürünleri farklı açılardan görüntüler. OpenCV kullanılarak alınan görüntüler, YOLO modeli ile işlenir. YOLO modeli, tespit edilen ürünlerin koordinatlarını ve sınıflarını belirler. Tespit edilen ürünlerin boyutları, kamera kalibrasyon değerleri kullanılarak piksel cinsinden hesaplanır ve santimetreye dönüştürülür.

5. 3.2 Veritabanı Modülü

Veritabanı Modülü, Django model sınıfı olarak tanımlanan Measurement modelini içerir. Measurement modeli, ürün ID'si, ürün adı, kamera 1 ve kamera 2 tarafından ölçülen genişlik ve yükseklik bilgilerini ve zaman damgasını içerir. Bu model, tespit edilen ürünlerin bilgilerini saklamak için kullanılır ve veritabanında tablolar halinde tutulur.

5. 3.3 Kullanıcı Arayüzü Modülü

Kullanıcı Arayüzü Modülü, Django templateları kullanılarak oluşturulmuş HTML/CSS sayfalarından oluşur. Bu sayfalar, kullanıcı dostu arayüzler sunar ve kullanıcıların sistemle etkileşimini sağlar. Arayüz, tespit edilen ürünlerin bilgilerini dinamik olarak gösteren tablolar

içerir. Ayrıca, kullanıcı kayıt ve giriş işlemleri için formlar ve doğrulama mekanizmaları da bu modül tarafından sağlanır.

5. 4. Veri Tabanı Tasarımı

5. 4.1 Measurement Modeli

Measurement modeli, ürün ID'si, ürün adı, kamera 1 ve kamera 2 tarafından ölçülen genişlik ve yükseklik bilgileri ve zaman damgası alanlarından oluşur. Bu model, tespit edilen ürünlerin tüm detaylarını içerir ve veritabanında saklanır. Ürün ID'si, her ürün için benzersiz bir tanımlayıcıdır. Ürün adı, tespit edilen ürünün adını içerir. Kamera 1 ve kamera 2 genişlik ve yükseklik alanları, ürünün her iki kameradan ölçülen boyutlarını içerir. Zaman damgası, ölçümün yapıldığı zamanı belirtir.

5.5. Kullanıcı Arayüzü Tasarımı

5.5.1 Ana Sayfa (index.html)

Ana sayfa, gerçek zamanlı ürün bilgilerini gösteren bir tablo içerir. Bu tabloda, tespit edilen ürünlerin ID'si, adı, her iki kameradan ölçülen genişlik ve yükseklik bilgileri ve zaman damgası gösterilir. Ayrıca, kamera görüntülerini gösteren bir alan da bulunmaktadır. Kullanıcılar, "Başlat" ve "Durdur" butonları ile sistemin çalışmasını kontrol edebilirler.

5.5.2 Kayıt ve Giriş Sayfaları (register.html, login.html)

Kayıt ve giriş sayfaları, kullanıcıların sisteme kayıt olmalarını ve giriş yapmalarını sağlar. Kayıt sayfasında, kullanıcı adı ve şifre gibi bilgilerin girildiği formlar bulunmaktadır. Giriş sayfası, kayıtlı kullanıcıların sisteme giriş yapmalarını sağlar. Hatalı girişlerde, kullanıcıya hata mesajları gösterilir ve tekrar denemeleri istenir.

5.6. Kullanım Durumları

5.6.1 Ürün Tespiti ve Boyutlandırma

Kullanıcı, sistemi başlatarak ürün tespiti ve boyutlandırma işlemlerini başlatır. Kameralar, konveyör bant üzerindeki ürünleri görüntüler ve YOLO modeli, ürünleri tespit eder. Tespit edilen ürünlerin boyutları hesaplanır ve veritabanına kaydedilir. Bu süreç, kullanıcı arayüzünde gerçek zamanlı olarak gösterilir ve kullanıcı, tespit edilen ürünlerin bilgilerini tabloda görebilir.

5.6.2 Kullanıcı Kayıt ve Giriş

Kullanıcı, kayıt sayfasından yeni bir hesap oluşturur. Kayıt işlemi tamamlandıktan sonra, kullanıcı giriş sayfasından sisteme giriş yapar. Başarılı girişten sonra, kullanıcı ana sayfaya yönlendirilir ve sistemin diğer işlevlerine erişebilir. Kayıt ve giriş işlemleri, kullanıcı bilgilerini doğrulamak ve güvenli erişim sağlamak için tasarlanmıştır.

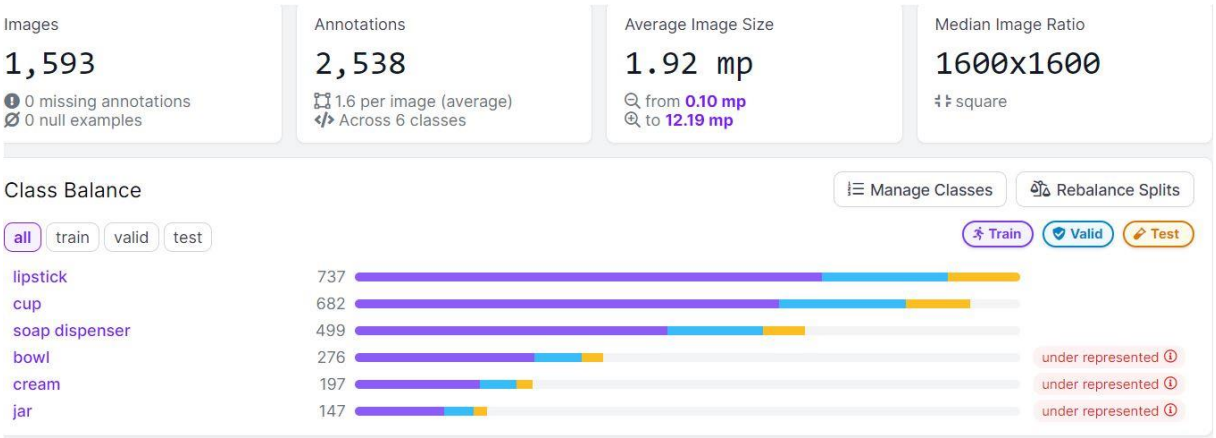
6. PROJENİN GERÇEKLEŞTİRİLMESİ

6.1. Verisetinin Oluşturulması

Projede , görüntü işleme tekniklerini kullanarak konveyor banttan geçen nesneleri tespit etmek ve boyutlarını belirlemek amacıyla YoloV8 ve OpenCV teknolojilerinden yararlanıldı. Bu hedef doğrultusunda, öncelikle verisetini oluşturma aşamasına odaklanıldı. Projenin veriseti altı farklı sınıftan oluşmaktadır:

- Bowl (kase)
- Soap Dispenser (sabunluk)
- Cream (krem kutusu)
- Cup (bardak)
- Lipstick (ruj)
- Jar (kavanoz)

Bu sınıflara ait fotoğraf dağılımı aşağıdaki şekil 2'de verilmiştir. Verisetinde lipstick, soap dispenser ve cup sınıflarının fazla olmasının sebebi projede bu üç sınıfın fotoğraflarının verisetinde artırılmasıdır. Bunun sebebi başlangıçta altı sınıf için geliştirilen verisetinin, projenin ilerleyen zamanlarında bu üç sınıf için genişletilerek daha yüksek performans elde etmek oldu. Bu genişletme proje daha da geliştirilmek istendiğinde diğer sınıflar için de yapılabilir.



Şekil 2. Sınıfların verisetindeki dağılımı

6.1.1. Veri Toplama ve Etiketleme Süreci

Verisetini hazır bir veri seti kullanarak oluşturmak yerine, projeye özgü ve daha yüksek doğrulukta sonuçlar elde edebilmek için tüm sınıflara ait nesnelerin fotoğraflarını kendimiz çektik. Çekim sürecinde, her bir nesnenin farklı açılardan, farklı ışıklarda ve çeşitli pozisyonlarda fotoğrafları çekildi. Bu sayede, modelin gerçek dünyadaki çeşitli durumları daha iyi anlaması ve tespit edebilmesi amaçlandı.

Toplanan fotoğrafların etiketlenmesi için Roboflow platformunu kullandık. Roboflow, etiketleme sürecini kolaylaştıran ve hızlandıran kullanıcı dostu bir arayüze sahiptir.

Her bir fotoğrafı tek tek inceleyerek, ilgili nesneler doğru bir şekilde etiketlendi. Roboflow'un sağladığı çeşitli avantajlar sayesinde veri etiketleme süreci oldukça verimli ve etkili bir şekilde gerçekleştirilmiştir. Ayrıca, Roboflow veri kümesinin kalitesini artırmak için çeşitli veri iyileştirme ve veri genişletme (augmentation) teknikleri sunar, bu da modelin genelleme yeteneğini artırarak farklı koşullar altında daha iyi performans göstermesini sağlar. Bu işlem, modelin eğitim aşamasında nesneleri doğru bir şekilde tanımlayabilmesi için kritik bir öneme sahiptir.

6.1.2. Veri Setinin Dağılımı

Roboflow kullanılarak oluşturulan veri seti , toplamda 4000 fotoğraftan oluşmaktadır. Bu fotoğraflar, eğitim, doğrulama ve test setlerine belirli oranlarda dağıtılmıştır:

- **Eğitim Seti (Train Set)**: 3342 fotoğraf
- **Doğrulama Seti (Validation Set)**: 318 fotoğraf
- **Test Seti (Test Set)**: 161 fotoğraf

Eğitim seti, modelin öğrenme süreci için kullanılırken, doğrulama seti modelin performansını değerlendirmek ve hyperparametre ayarlarını yapmak için kullanılmıştır. Test seti ise, modelin gerçek dünya verileri üzerindeki performansını bağımsız olarak ölçmek için ayrılmıştır.

6.1.3. Ön İşleme ve Veri Artırımı (Augmentation)

Roboflow platformunda, veri setini daha etkili bir şekilde kullanabilmek için çeşitli ön işleme (preprocessing) ve veri artırımı (augmentation) tekniklerinden yararlanıldı:

- Auto-Orient: Tüm görüntüler, doğru yönde hizalanmaları için otomatik olarak yönlendirildi.
- Resize: Görseller, modelin girdi boyutlarına uygun hale getirilmesi için 640x640 piksel boyutuna ölçeklendirildi.
- Outputs per Training Example: Her bir eğitim örneği için 3 farklı çıktı üretildi.
- Saturation: Doygunluk değeri, -%27 ile +%27 arasında değiştirildi.
- Exposure: Pozlama değeri, -%10 ile +%10 arasında değiştirildi.
- Blur: Görüntülere 2.5 piksele kadar bulanıklık eklendi.
- Noise: Görüntülere, toplam piksellerin %0.1'ine kadar gürültü eklendi.

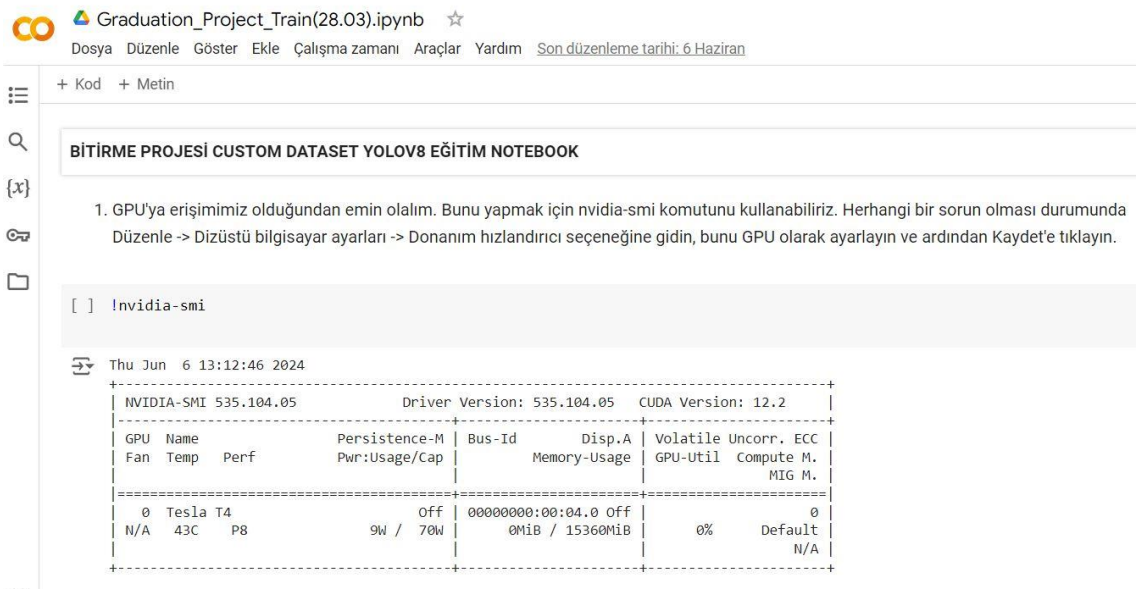
Bu ön işleme ve veri artırımı teknikleri, modelin eğitim sürecinde daha çeşitli ve genelleştirilmiş verilerle karşılaşmasını sağlayarak, modelin performansını ve doğruluğunu artırmaya yönelik önemli adımlar olarak kullanılmıştır.

6.2. Veriseti Eğitimi Ve Model Oluşturma

6.2.1. Veriseti Eğitimi Kodları

Veri seti oluşturulduktan sonra model, YOLOv8 ile Google Colab kullanılarak eğitilmiştir. Eğitim süreci 27 epoch boyunca sürdürülmüş ve model performansını çeşitli epochlarda değerlendirerek sonuçlar kıyaslanmıştır. Değelendirmeye geçilmeden önce aşağıda colabta yazılan eğitim kodları ve açıklamaları verilmiştir.

- Aşağıdaki kod bloğu, sistemde bir GPU olup olmadığını kontrol etmek için kullanılır. nvidia-smi komutu, NVIDIA GPU'ların durumunu ve özelliklerini gösterir.



- Aşağıdaki kod bloğu, ultralytics paketinin belirli bir sürümünü (8.0.196) yükler. Display.clear_output() ile önceki çıktı temizlenir ve ultralytics modülü import edilir ve ultralytics.checks() ile kurulum ve sistem kontrolleri yapılır.
- Üçüncü başlıktaki kod bloğu ile ultralytics paketinden YOLO sınıfı içe aktarılır ve diğer IPython display özellikleri için gerekli modüller import edilir.



- Aşağıdaki kod bloğu, Roboflow kullanarak veri kümesi indirme ve hazırlama işlemlerini yapar. İlk iki komut, veri kümesinin indirileceği dizini oluşturur ve bu dizine geçiş yapar. Roboflow API'si kullanılarak projeye erişilir ve belirli bir sürümdeki veri kümesi indirilir (yolov8 formatında).

4. **Exporting Dataset** using Roboflow code

```
[ ] !mkdir {HOME}/datasets
    %cd {HOME}/datasets


    !pip install roboflow --quiet

    !pip install roboflow

    from roboflow import Roboflow
    rf = Roboflow(api_key="CfJBeOJUSFzKrWHcFyro")
    project = rf.workspace("cagla-bhxzs").project("graduationproject-iy9cg")
    version = project.version(14)
    dataset = version.download("yolov8")
```

- Aşağıdaki kod bloğu, YOLOv8 kullanarak özel bir veri kümesi ile nesne tespiti modeli eğitimi yapmak için kullanılır. İlk olarak, çalışma dizini HOME değişkeni tarafından temsil edilen dizine değiştirilir. Ardından, yolo komutu ile YOLOv8 eğitim süreci başlatılır. task=detect ile nesne tespiti görevi seçilir, mode=train ile eğitim modu ayarlanır, model=yolov8s.pt ile önceden eğitilmiş bir model kullanılır, data={dataset.location}/data.yaml ile veri kümesinin yapılandırma dosyası belirtilir, epochs=27 ile 27 epoch boyunca eğitim yapılır, imgsz=800 ile görüntü boyutu 800x800 piksel olarak ayarlanır ve plots=True ile eğitim sırasında çeşitli görseller ve grafikler oluşturulur. Bu süreç, modelin performansını ve ilerlemesini görselleştirerek takip etmeyi sağlar.

5. **Custom Training**

```
%cd {HOME}

!yolo task=detect mode=train model=yolov8s.pt data={dataset.location}/data.yaml epochs=27 imgsz=800 plots=True
```

```
/content
New https://pypi.org/project/ultralytics/8.2.28 available 😃 Update with 'pip install -U ultralytics'
Ultralytics YOLOv8.0.196 🚀 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
engine/trainer: task=detect, mode=train, model=yolov8s.pt, data=/content/datasets/GraduationProject-14/data.yaml, epochs=
2024-06-06 14:25:22.150084: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261] Unable to register cuDNN fact
2024-06-06 14:25:22.150142: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607] Unable to register cuFFT facto
2024-06-06 14:25:22.152030: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515] Unable to register cuBLAS fa
Overriding model.yaml nc=80 with nc=6

                   from  n    params  module                                       arguments
  0                  -1  1       928  ultralytics.nn.modules.conv.Conv             [3, 32, 3, 2]
  1                  -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
  2                  -1  1     29056  ultralytics.nn.modules.block.C2f             [64, 64, 1, True]
  3                  -1  1     73984  ultralytics.nn.modules.conv.Conv             [64, 128, 3, 2]
  4                  -1  2    197632  ultralytics.nn.modules.block.C2f             [128, 128, 2, True]
  5                  -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  6                  -1  2    788480  ultralytics.nn.modules.block.C2f             [256, 256, 2, True]
```

- Aşağıdaki kod bloğu, özel nesne tespiti modelini doğrulamak için kullanılır. Çalışma dizini HOME olarak ayarlanır ve ardından yolo komutu ile tahmin süreci başlatılır. task=detect ile nesne tespiti, mode=predict ile tahmin modu, model={HOME}/runs/detect/train/weights/best.pt ile en iyi model, conf=0.25 ile minimum %25 güven oranı, source={dataset.location}/test/images ile test görüntüleri dizini ve save=True ile sonuçların kaydedilmesi sağlanır. Bu, modelin test verisindeki performansını değerlendirir ve sonuçları kaydeder.



6.2.2. Modelin Performans Metrikleri Değerlendirilmesi

Eğitimde 27 Epoch kullanılması deneme yanılma ile performans metriklerinin karşılaştırılması ile optimize edilmiş ve en iyi performans sağlayan Epoch sayısı olarak belirlenmiştir. 27 epochta performans metriklerini inceleyecek olursak:

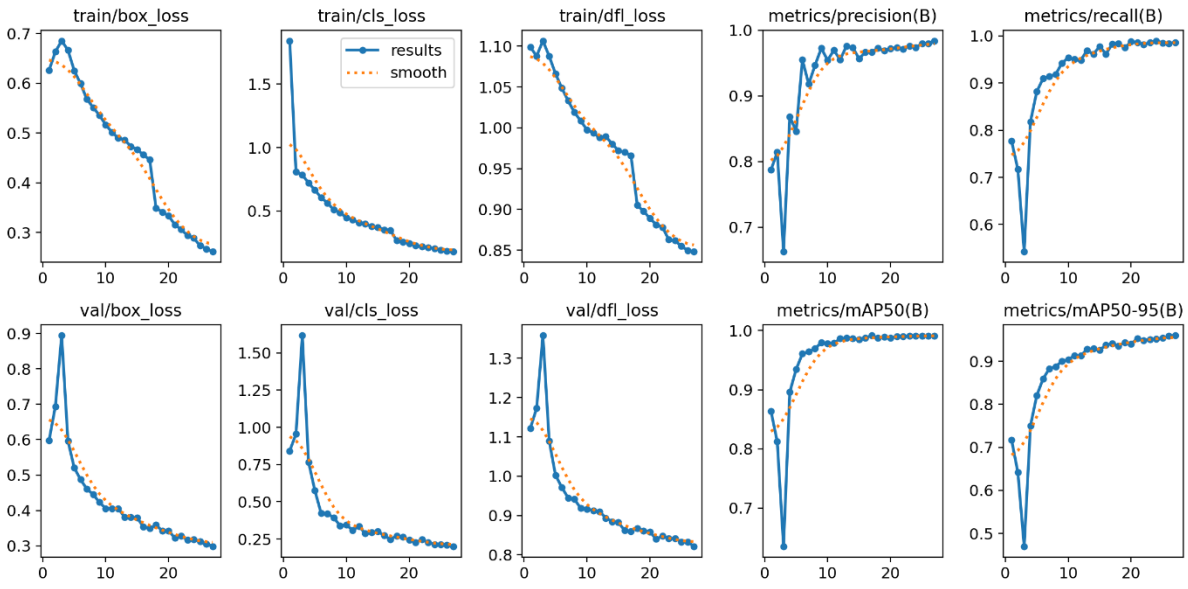
1. **Başlangıç Performansı (Epoch 1-3):**
    - **Epoch 1:** Başlangıçta Box (P) = 0.787, R (Recall) = 0.777, mAP50 = 0.864, mAP50-95 = 0.717.
    - **Epoch 2:** Box (P) = 0.814, R = 0.717, mAP50 = 0.812, mAP50-95 = 0.642. Precision biraz artarken recall ve mAP düşmüş.
    - **Epoch 3:** Box (P) = 0.663, R = 0.542, mAP50 = 0.635, mAP50-95 = 0.47. Performans düşüşü, modelin henüz öğrenme sürecinde olduğunu gösteriyor.
2. **Orta Dönem Performansı (Epoch 4-14):**
    - **Epoch 4-7:** Box (P) ve R önemli ölçüde artış gösteriyor. Epoch 7'de Box (P) = 0.918, R = 0.914, mAP50 = 0.964, mAP50-95 = 0.883.
    - **Epoch 8-14:** Performans stabil bir şekilde artıyor. Epoch 14'de Box (P) = 0.974, R = 0.961, mAP50 = 0.987, mAP50-95 = 0.929. Bu dönemde kayıpların azaldığı ve modelin daha iyi öğrenme gerçekleştirdiği gözlemleniyor.
3. **Son Dönem Performansı (Epoch 15-27):**
    - **Epoch 15-21:** Performans daha da iyileşiyor ve model neredeyse maksimum doğruluğa ulaşıyor. Epoch 21'de Box (P) = 0.974, R = 0.986, mAP50 = 0.99, mAP50-95 = 0.953.
    - **Epoch 22-27:** Model performansını stabilize etmiş durumda. Epoch 27'de Box (P) = 0.984, R = 0.986, mAP50 = 0.991, mAP50-95 = 0.96. Bu, modelin artık optimize olduğunu ve yüksek doğrulukta sonuçlar verdiğini gösteriyor.

Spesifik Sınıfların Performansı (Epoch 27)

- **Genel Performans:**
  - Precision (P) = 0.984
  - Recall (R) = 0.986
  - mAP50 = 0.991
  - mAP50-95 = 0.96
- **Sınıf Bazında:**
  - **Bowl:** P = 0.993, R = 1, mAP50 = 0.995, mAP50-95 = 0.972.
  - **Cream:** P = 0.975, R = 0.963, mAP50 = 0.989, mAP50-95 = 0.94.
  - **Cup:** P = 1, R = 0.993, mAP50 = 0.995, mAP50-95 = 0.977.
  - **Jar:** P = 0.959, R = 1, mAP50 = 0.993, mAP50-95 = 0.987.
  - **Lipstick:** P = 0.997, R = 0.979, mAP50 = 0.989, mAP50-95 = 0.939.
  - **Soap Dispenser:** P = 0.981, R = 0.983, mAP50 = 0.986, mAP50-95 = 0.945.

Kayıp Metrikleri:

- **Box_loss:** Başlangıçta yüksekken zamanla azalmış. Son epoch'ta 0.2617.
- **Cls_loss:** Benzer şekilde başlangıçta yüksek, zamanla azalmış ve son epoch'ta 0.1796.
- **Dfl_loss:** İlk epoch'ta 1.099 iken son epoch'ta 0.8482.



Şekil 3. 27 Epoch performans metrikleri grafikleri



Şekil 4. Modelin yaptığı tahminler

Yukarıdaki şekil 4 modelin validasyon veri kümesindeki bir batch üzerinde yaptığı tahminleri gösteriyor. "val_batch0_pred" ifadesi, validasyon işlemi sırasında ilk batch'teki (batch 0) görüntüler üzerindeki model tahminlerini temsil eder.

Epoch Verileri ve Performans Kıyaslaması:

Tablo 1. Epoch sayılarına göre performans metrikleri

Epoch	Box P	Box R	mAP50	mAP50-95	Learning Rate
25	0.971	0.996	0.990	0.951	0.000157
27	0.977	0.986	0.991	0.957	0.000083
30	0.981	0.993	0.993	0.964	0.000070

- 27 ve 30. epoch'ta Precision (0.981) ve Recall (0.993) değerleri, 25 epoch değerlerine kıyasla biraz daha yüksek. Bu, modelin doğru pozitif tespitleri artırırken yanlış negatif tespitleri azalttığını gösterir. Ayrıca mAP50 ve mAP50-95 değerleri de yüksek seviyede. Bu, modelin genel doğruluğunun ve farklı IoU eşiklerinde tutarlılığının arttığını gösterir.
- 30. epoch'a kadar modelin performansının sürekli iyileştiği görülüyor. Özellikle, mAP50-95 değerindeki sürekli artış, modelin genel performansının iyileştiğini ve daha doğru tahminler yaptığını gösterir.
- 27 ve 30 Epochta performans metrikleri yüksek 30 da ise 27 den az da olsa daha yüksek olduğu görülüyor fakat learning rate 30 Epoch da daha da düştüğü için burada modelde ezberleme söz konusu olduğundan projede denendiğinde nesne tespitinde daha düşük

başarıya sahip olduğu gözlemlendi. Bu sebeple 27 Epoch optimum değer olarak belirlendi ve model 27 Epoch'ta eğitilerek yüksek verim alındı.

Sonuç olarak, model 27 epoch sonunda oldukça yüksek bir doğruluk ve genel performans metrikleri elde etmiş. Kayıp metrikleri azalmış ve performans metrikleri (precision, recall, mAP50, mAP50-95) sürekli iyileşme göstermiş. Eğitim süreci, modelin başarıyla optimize olduğunu ve iyi bir genel performansa ulaştığını gösteriyor. Bu veriler ile , modelin oldukça etkili bir şekilde eğitildiğini ve görüntü işleme görevinde yüksek performans gösterdiğini söyleyebiliriz.

6.3. Görüntü İşleme Algoritması

Model eğitimi tamamlandıktan sonra, kendi veri setimize göre eğitilen ve oluşturulan model, kameralardan alınan görüntüleri işlemek amacıyla kullanılmıştır. Bu görüntü işleme algoritması, kameraların gerçek zamanlı olarak çektiği ürünleri tespit etmek, sınıflandırmak ve boyutlarını belirlemek amacıyla yazılmıştır. Nesne tespiti YoloV8 ile yapılırken, görüntü üzerindeki işlemler OpenCV kütüphanesi kullanılarak gerçekleştirilmiştir. Nesnelerin boyutlarını belirlerken bir referans obje ile kalibrasyon yapılmıştır. Referans objenin piksel cinsinden genişlik ve yükseklik değerleri kameralar ile belirlenmiş, ardından gerçek dünya boyutlarına çevirmek için referans objenin santimetre cinsinden boyutları ölçülmüş ve bir oran oluşturulmuştur. Bu oran kullanılarak diğer ürünlerin boyutları doğru bir şekilde ölçülmüştür. Bu işlemi yaparken kritik nokta, kamera ile konveyör banttan geçen ürünler arasındaki mesafeyi sabit tutarak, piksel cinsinden boyutlarının değişmeden, düşük hata payı ile doğru bir şekilde ölçülmesini sağlamaktır. Kameralar, bu durumu göz önünde bulundurarak konveyör bantta uygun yerlere sabitlenmiştir. Bu amaç için yazılan algıritmanın kodları aşağıda ayrıntılı olarak verilmiştir.

```python
@login_required
def process_cameras(request):
    if request.method == 'POST':
        if 'start' in request.POST:
            # YOLO modelini yükle
            model = YOLO("27Epoch.pt")

            # Kameraları başlat
            kamera1 = cv2.VideoCapture(0)
            kamera2 = cv2.VideoCapture(1)

            font = cv2.FONT_HERSHEY_SIMPLEX

            # Piksel/cm dönüşüm oranları
            camera1_ratiow_px_cm = 289 / 11.2
            camera1_ratioh_px_cm = 253 / 9.2
            camera2_ratiow_px_cm = 277 / 11
            camera2_ratioh_px_cm = 226 / 7

            # Geçici ürün bilgileri saklanacak dictionary
            current_product = None
            last_detection_time = None
```

Şekil 5. Görüntü işleme algoritması kod-1

Şekil 5'de process_cameras fonksiyonunun başlangıcı ve bazı önemli yapılandırma adımları gösterilmektedir. YOLO modeli yüklenir (27Epoch.pt) ve iki kamera başlatılır (kamera1 ve kamera2). Pikselden santimetreye dönüşüm oranları tanımlanır ve tespit edilen ürünlerin geçici olarak saklanacağı değişkenler tanımlanır. Bu kod parçaları, YOLO modeli kullanarak nesne tespiti yaparken OpenCV ile gerçek zamanlı görüntü işlemlerini ve ölçümlerini nasıl gerçekleştirdiğini göstermektedir. Bu işlemler, kameraların sabitlenmesi ve referans nesneler kullanılarak kalibrasyon yapılması ile hassas ölçümler sağlar.

```
current_time = time.time()
# Birinci kameradan ürün bilgilerini al
for i in range(len(results1[0].boxes)):
    x1, y1, x2, y2 = results1[0].boxes.xyxy[i]
    score = results1[0].boxes.conf[i]
    label = results1[0].boxes.cls[i]
    x1, y1, x2, y2, score, label = int(x1), int(y1), int(x2), int(y2), float(score), int(label)
    name = results1[0].names[label]
    if score < 0.5:
        continue

    width_px = x2 - x1
    height_px = y2 - y1

    width_cm = round(width_px / camera1_ratiow_px_cm, 2)
    height_cm = round(height_px / camera1_ratioh_px_cm, 2)

    cv2.rectangle(img1, (x1, y1), (x2, y2), (255, 0, 0), 2)
    text = f"{name} {score:.2f} ({width_cm}cm x {height_cm}cm)"
    cv2.putText(img1, text, (x1, y1 - 10), font, 0.7, (255, 0, 255), 2)
```

Şekil 6. Görüntü işleme algoritması kod-2

Şekil 6'da birinci kamera ile ilgili nesne tespiti ve ölçüm işlemleri detaylandırılmaktadır. Tespit edilen her bir nesne için results1 üzerinden geçilerek her bir nesnenin koordinatları, güven puanı ve sınıf etiketi alınır. Güven puanı 0.5'in altında olan nesneler atlanır. Tespit edilen nesnenin genişlik ve yüksekliği piksel cinsinden hesaplanır ve ardından santimetreye dönüştürülür. Tespit edilen nesne, görüntü üzerinde dikdörtgen ile çizilir ve nesne bilgileri yazılır.

```
# Geçici ürün bilgileri saklanacak dictionary
current_product = None
last_detection_time = None

while True:
    # Birinci kameradan bir kare yakala
    ret1, img1 = kamera1.read()
    # İkinci kameradan bir kare yakala
    ret2, img2 = kamera2.read()

    if ret1:
        # İlk kareyi YOLO modeline ver ve tespitleri yap
        results1 = model(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB), verbose=False)
    else:
        img1 = cv2.putText(cv2.UMat(480, 640, cv2.CV_8UC3, (0, 0, 255)), "Camera 1 Error", (50, 50), font, 1, (255, 255,
        results1 = []

    if ret2:
        # İkinci kareyi 90 derece döndür
        img2 = cv2.rotate(img2, cv2.ROTATE_90_CLOCKWISE)
        # İkinci kareyi YOLO modeline ver ve tespitleri yap
        results2 = model(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB), verbose=False)
    else:
        img2 = cv2.putText(cv2.UMat(480, 640, cv2.CV_8UC3, (0, 0, 255)), "Camera 2 Error", (50, 50), font, 1, (255, 255,
        results2 = []
```

Şekil 7. Görüntü işleme algoritması kod-3

Şekil 7’de iki kameradan kareler yakalayarak YOLO modeline vermekte ve nesne tespiti yapmaktadır. İlk olarak, kamera1 ve kamera2'den kareler yakalanır ve bu kareler YOLO modeline verilerek nesne tespiti yapılır. Eğer kare yakalama başarılı olmazsa, hata mesajı görüntülenir. Tespit edilen nesneler üzerinden geçilerek her bir nesnenin koordinatları, güven puanı ve sınıf etiketi alınır. Daha sonra, tespit edilen nesneler görüntü üzerinde dikdörtgenler ile çizilir ve bilgileri yazılır. İki kamera görüntüsü yan yana birleştirilir ve gösterilir.

```
        if current_product is None or current_time - last_detection_time > 2:
            product_id = generate_unique_product_id()
            current_product = {
                "product_id": product_id,
                "product_name": name,
                "camera1_width": width_cm,
                "camera1_height": height_cm,
                "timestamp": current_time
            }
            last_detection_time = current_time

    # İkinci kameradan ürün bilgilerini al
    for i in range(len(results2[0].boxes)):
        x1, y1, x2, y2 = results2[0].boxes.xyxy[i]
        score = results2[0].boxes.conf[i]
        label = results2[0].boxes.cls[i]
        x1, y1, x2, y2, score, label = int(x1), int(y1), int(x2), int(y2), float(score), int(label)
        name = results2[0].names[label]
        if score < 0.5:
            continue
```

Şekil 8. Görüntü işleme algoritması kod-4

Şekil 8’de bir nesne algılama sürecinde ürün bilgilerini güncellemek ve yeni bir ürün algılandığında yeni bir ürün bilgisi oluşturmak için kullanılır. Eğer current_product boş veya son algılama süresinden 2 saniye geçmişse, yeni bir ürün oluşturulur. Bu işlem, generate_unique_product_id fonksiyonu ile benzersiz bir ürün kimliği oluşturulması ve current_product sözlüğünün ürün kimliği, adı, kamera1 genişlik ve yükseklik bilgileri ve zaman damgası ile güncellenmesiyle yapılır. Ayrıca, last_detection_time güncellenir. results2 içindeki kutular (boxes) döngü ile gezilerek her kutu için koordinatlar, skor ve etiket bilgileri alınır. Eğer skor 0.5'ten küçükse, döngü devam ettirilir. Alınan koordinatlar ve skorlar, int ve float dönüşümleri ile belirli değerlere atanır. Nesnenin adı, results2 içindeki etiketler ile belirlenir.

```
        width_px = x2 - x1
        height_px = y2 - y1

        width_cm = round(width_px / camera2_ratiow_px_cm, 2)
        height_cm = round(height_px / camera2_ratioh_px_cm, 2)

        cv2.rectangle(img2, (x1, y1), (x2, y2), (255, 0, 0), 2)
        text = f"{name} {score:.2f} ({width_cm}cm x {height_cm}cm)"
        cv2.putText(img2, text, (x1, y1 - 10), font, 0.7, (255, 0, 255), 2)

        if current_product and current_product["product_name"] == name:
            current_product["camera2_width"] = width_cm
            current_product["camera2_height"] = height_cm
```

Şekil 9. Görüntü işleme algoritması kod-5

Şekil 9’da, bir nesnenin boyutlarını hesaplayıp görüntü üzerinde bu bilgileri çizmek ve ekrana yazdırmak için kullanılır. Öncelikle, x1, y1, x2 ve y2 koordinatları arasındaki piksel cinsinden genişlik ve yükseklik hesaplanır. Bu piksel cinsinden genişlik ve yükseklik, belirli bir oran kullanılarak santimetre cinsine çevrilir. cv2.rectangle fonksiyonu ile img2 üzerinde belirtilen koordinatlar arasında kırmızı bir dikdörtgen çizilir. cv2.putText fonksiyonu ile nesnenin adı, skoru ve boyutları metin olarak dikdörtgenin üzerine yazılır. Eğer mevcut ürün ve algılanan ürün adı eşleşiyorsa, current_product sözlüğü içinde genişlik ve yükseklik bilgileri güncellenir.

```
        if img1.shape != img2.shape:
            height1 = min(img1.shape[0], img2.shape[0])
            width1 = int(img1.shape[1] * (height1 / img1.shape[0]))
            width2 = int(img2.shape[1] * (height1 / img2.shape[0]))

            img1 = cv2.resize(img1, (width1, height1))
            img2 = cv2.resize(img2, (width2, height1))

        combined_img = cv2.hconcat([img1, img2])
        cv2.imshow("Kameralar", combined_img)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

kamera1.release()
kamera2.release()
cv2.destroyAllWindows()
```

Şekil 10. Görüntü işleme algoritması kod-6

Şekil 10’da , iki farklı görüntüyü (img1 ve img2) birleştirmek için kullanılır. Öncelikle, img1 ve img2'nin boyutları kontrol edilir ve eğer farklıysa, bu görüntüler aynı boyuta getirilir. Bu işlem, her iki görüntünün de minimum yüksekliğinin belirlenmesi ve genişliklerinin bu yüksekliğe göre oranlanarak hesaplanmasıyla yapılır. cv2.resize fonksiyonu kullanılarak img1 ve img2 yeniden boyutlandırılır. Yeniden boyutlandırılan img1 ve img2, cv2.hconcat fonksiyonu ile yatay olarak birleştirilir ve cv2.imshow fonksiyonu ile ekranda gösterilir. Kullanıcı 'q' tuşuna bastığında, döngü kırılarak program sonlandırılır. Son olarak, kamera1.release() ve kamera2.release() ile kamera kaynakları serbest bırakılır ve cv2.destroyAllWindows() ile tüm OpenCV pencereleri kapatılır.

6.4. Kullanıcı Arayüzü Ve Veri tabanı

Kullanıcı arayüzü HTML, CSS ve Django framework'ü kullanılarak geliştirilmiştir. Veri tabanı yönetimi için ise Django'nun varsayılan olarak sunduğu SQLite veritabanı kullanılmıştır.

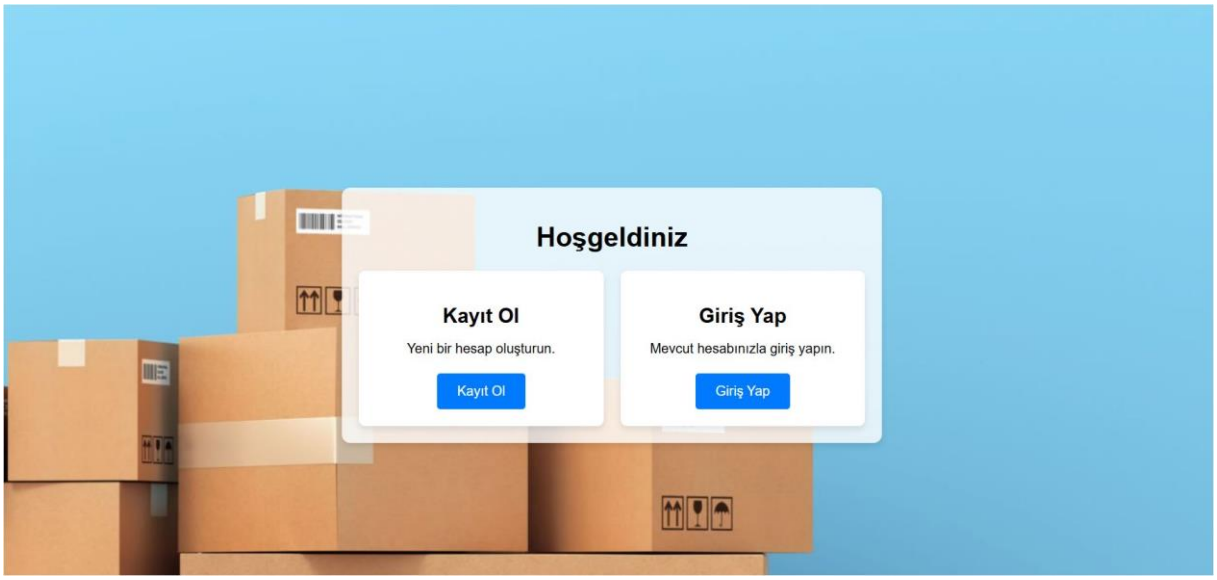
6.4.1. Kullanıcı Arayüzü

Projenin kullanıcı arayüzüne bakıldığında anasayfada hoşgeldin sayfası ile kayıt ol ve giriş seçenekleri bulunmaktadır. Kayıt olan kullanıcı giriş yapar. Giriş yaptığı takdirde kameraları başlatacağı arayüze gelir ve burada tespit edilen nesnelerin bilgilerine erişilir.

Bu bilgiler aynı zamanda Django Administration tarafından sağlanan bir arayüz ile veri tabanında gözlemlenir ve üzerinde değişiklik yapılabilir. Fakat veri tabanı sadece django projesinde oluşturulan admin tarafından gözlemlenir. Arayüz kullanıcıların etkileşimde bulunabileceği basit ve sezgisel bir tasarım sunmak amacıyla HTML ve CSS kullanılarak oluşturulmuştur.

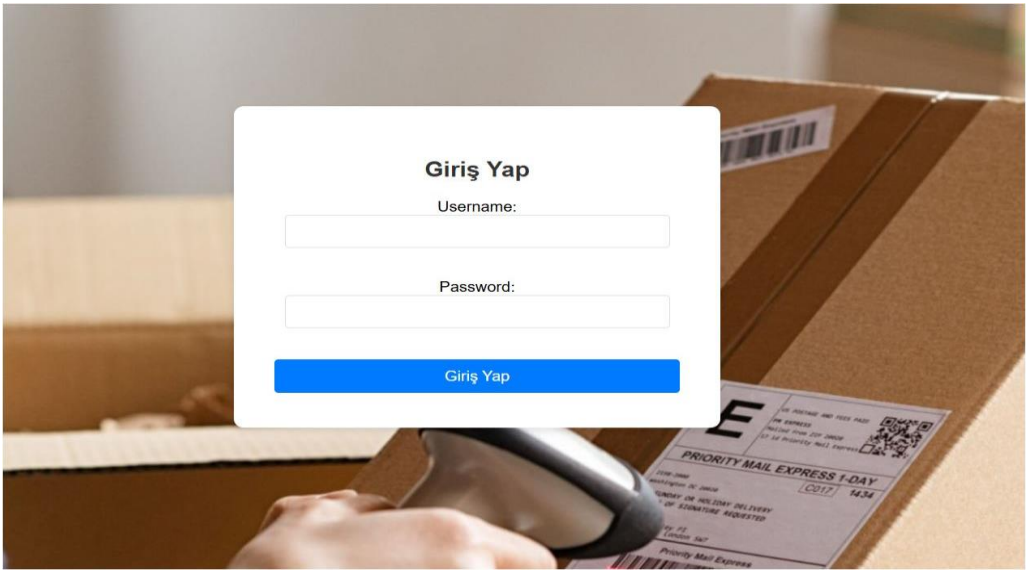
Django, bu HTML ve CSS dosyalarını dinamik olarak sunmak ve kullanıcı isteklerini işlemek için kullanılmıştır. Django'da, her bir sayfa için ayrı HTML şablonları oluşturulur ve bu şablonlar templates dizini altında saklanır. CSS dosyaları, Django'da genellikle static dizini altında saklanır ve HTML şablonlarına linklenir. Django'nun şablon sistemi, dinamik HTML sayfaları oluşturmak için kullanılır. Veri tabanından alınan veriler, Django şablon motoru tarafından işlenir ve HTML şablonlarına yerleştirilir. Böylece kullanıcıya dinamik ve güncel içerik sunulur.

Arayüz bahsedildiği gibi her eğitim seviyesi ve bilgisayar kullanma bilgisine uygun olması açısından kullanıcı dostu bir şekilde basitçe tasarlanmıştır. Aşağıdaki şekil 11'de arayüz açıldığında hoş geldiniz sayfası görülmektedir. Burada operatör kullanıcı adı ve şifresiyle kaydolarak daha sonrasında giriş yapılabilir.

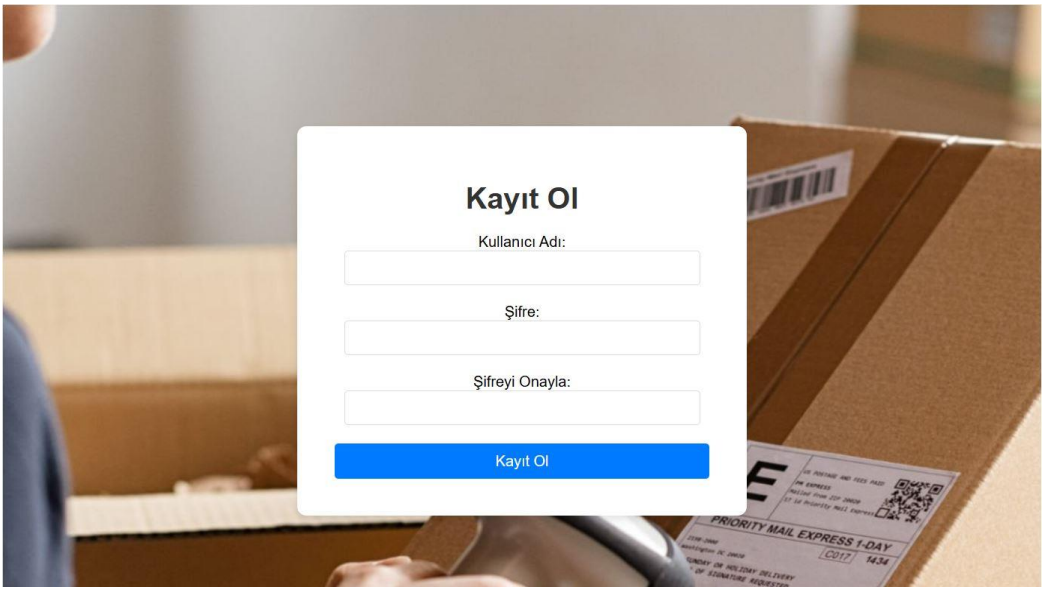


Şekil 11. Arayüz "hoşgeldiniz" sayfası

Aşağıdaki kayıt ol ve giriş sayfalarından ise operatörler sisteme giriş sağlayabilirler. Sisteme başarılı bir şekilde giriş yaptıkları takdirde ürünlerin bilgilerinin yer aldığı tablonun ve kameraların başlatılacağı butonun bulunduğu sayfaya erişim sağlayabilirler.

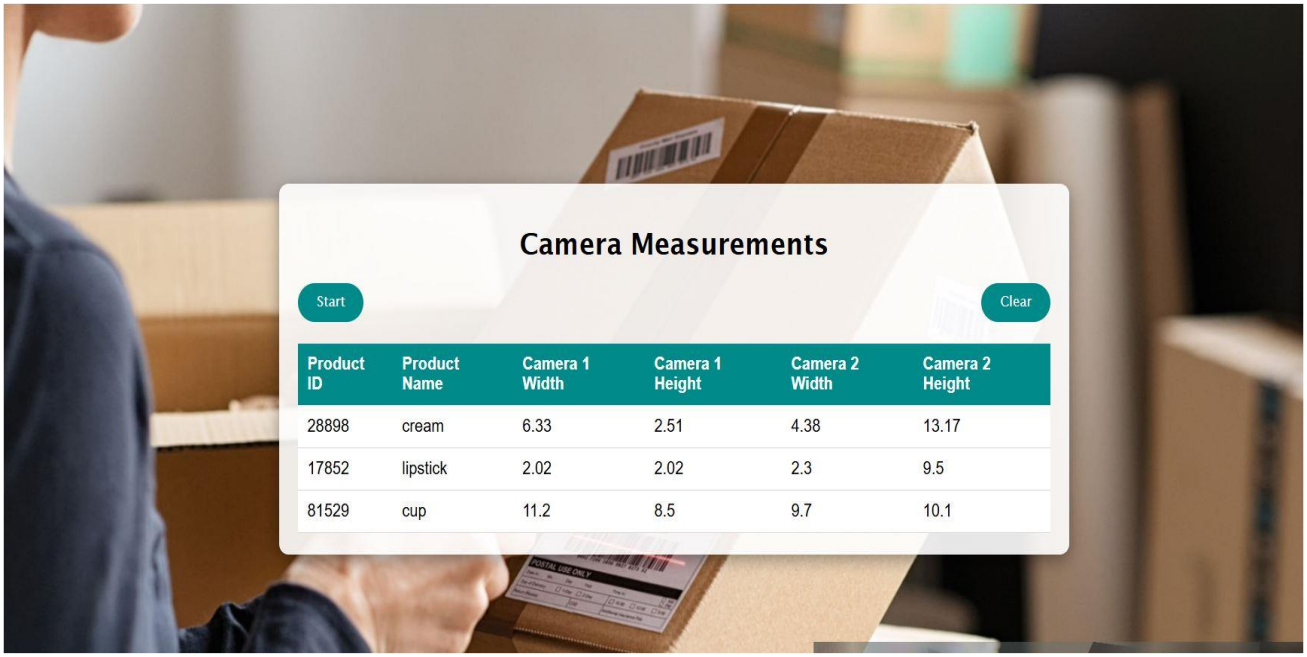


Şekil 12. Giriş yap sayfası



Şekil 13. Kayıt ol sayfası

Kullanıcı giriş yaptıktan sonra Şekil 14'deki ölçüm yapılan tablonun bulunduğu sayfaya yönlendirilir.



Şekil 14. Kameraların başlatıldığı ve ölçümlerin yazıldığı tablo

Yukarıdaki şekil 14'de gösterilen tabloda start butonu ila kameralar başlatılır ve görüntü işleme başlar. Tespit edilen ürünlere benzersiz bir productId atanır ve sınıf isimleri belirlenir. Daha sonra iki farklı kameradan gelen boyut bilgileri cm cinsinden ölçülür. Görüntü işleme ve kameralar q tuşuna basarak sistem durur ve belirlenen veriler bu tabloya yazılır. Tabloda fazla verilerin olması istenmediğinde "clear" butonu ile tablo temizlenir.

6.4.2.Veritabanı

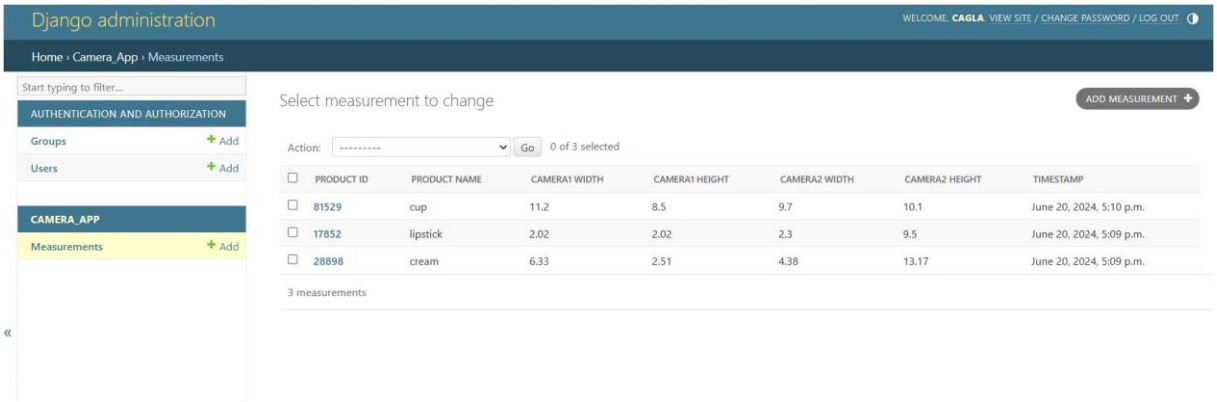
Projenin veri tabanı yönetimi için Django'nun varsayılan olarak sunduğu SQLite veri tabanı kullanılmıştır. SQLite, hafif, sunucusuz ve kurulumu kolay bir veri tabanı yönetim sistemidir. Django ile birlikte kullanıldığında, veri tabanı işlemleri oldukça basit ve etkilidir.

- **Model-View-Template (MVT) Yapısı:** Django, veri tabanı işlemlerini kolaylaştırmak için Model-View-Template yapısını kullanır. Model katmanı, veri tabanı tablolarını ve ilişkilerini temsil eder. Her bir model, bir veri tabanı tablosuna karşılık gelir ve bu tablonun alanları Python sınıfları ve öznitelikleri olarak tanımlanır. Bu modeller models.py dosyasında tanımlanır ve Django ORM (Object-Relational Mapping) tarafından yönetilir.

```
camera_project > camera_app > models.py > ...
1   from django.db import models
2
3   class Measurement(models.Model):
4       product_id = models.CharField(max_length=100)
5       product_name = models.CharField(max_length=100)
6       camera1_width = models.FloatField()
7       camera1_height = models.FloatField()
8       camera2_width = models.FloatField()
9       camera2_height = models.FloatField()
10      timestamp = models.DateTimeField(auto_now_add=True)
11
12      def __str__(self):
13          return f"{self.product_name} - {self.product_id}"
14
```

Şekil 15. Projenin models.py kodları

Aşağıdaki şekil 16'da tespiti yapılan nesnelerin Django administration arayüzünde veri tabanında gösterilmesi verilmiştir.



Şekil 16. Django Veri tabanı Arayüzü

6.5. Konveyör Bant Tasarımı

Kullanılan Malzemeler:

- Tahta
- Polymath merdane (sabit ve dönerli)
- 12V DC motor
- 12V adaptör
- 2 tane 1mm sacdan telefon koymak için sehpa
- 2 tane M8 gerdirme cıvatası
- 4 tane M8 somun
- 2 tane metal M8 ayar sıkma(kelebek) somun

- 1 tane plastik M8 sıkma kolu
- Bez bant
- 4 tane konveyör ayağı
- 2 tane 100 cm boy ve 14 cm eninde yan duvar tahta (kalınlık 10 mm)
- 1 tane yan duvarların ortasına boy 100 cm ve en 14 cm bağlantı tahtası

Yapılışı:

- Yan duvar tahtaları birbirine vidalandı.
- Önden sabit merdane için yatak deliği açıldı. Arkadan da döner merdane yatağı için yarıklı delik açıldı
- Açtıktan sonra tahtalar birbirinden ayrıldı. Üçüncü tahtayı da yan duvar tahtalarının ortasından vidalandı.
- Öndeki sabit merdane yataklara yerleştirildi. Arkadaki hareketli merdane arka tarafa yerleştirildi. Bez bant iki merdaneden geçirildi. Sıkma ayarlarından gerdirildi.
- Sabit merdanenin uç tarafına 12V şanzıman motoru yerleştirildi sabitlendi.
- Elektrik kablosuna 12V DC adaptör yerleştirildi. Kablo ve fiş takıldı.
- Telefon sehpaları da biri üstten görecek şekilde ortaya vidalandı. Diğeri yandan görecek şekilde arka uç tarafına vidalandı. 4 ayağa her uç kenarlarından vidalandı ve konveyör bant Şekil 17'deki gibi hazır hale geldi.



Şekil 17. Konveyör Bant

7. SONUÇ VE TARTIŞMA

Bu projede, konveyör banttan geçen nesneleri tespit etmek ve boyutlarını belirlemek için YOLOv8 ve OpenCV teknolojilerinin etkin bir şekilde kullanılabileceği gösterilmiştir. YOLOv8 ile eğitilen model, yüksek doğruluk ve genelleme yetenekleri sergileyerek nesne tespitinde başarılı olmuştur. Eğitim sürecinde veri artırma ve ön işleme tekniklerinin kullanılması, modelin performansını artırmış ve gerçek dünya koşullarında daha iyi sonuçlar elde edilmesini sağlamıştır. Gerçek zamanlı görüntü işleme algoritmaları ile nesnelerin boyutlarının referans obje kullanılarak doğru bir şekilde ölçülmesi, projenin hedeflerine ulaşmasında kritik bir rol oynamıştır. Konveyör bantta yapılan ölçüm sonuçları aşağıdaki şekillerde verilmiştir. Aşağıdaki şekil 18'de görüldüğü gibi lipstick %92 doğruluk oranında tespit edilmiş ve üst açıdan ilk kamera ile çekilen kısmın boyutları 2 cm olarak belirlenmiştir.



Şekil 18. Lipstick nesnesinin üst açıdan kamera ile tespiti ve boyutlandırılması

Aşağıdaki Şekil 19'da ise aynı ürünün yan açıdaki kamera ile tespit edilmiş ve boyutlarının belirlenmiş görüntüsü verilmiştir.



Şekil 19. Lipstick nesnesinin yan açıdan kamera ile tespiti ve boyutlandırılması

Aşağıdaki şekil 20 ve 21 de kameradan boyutları ölçülen lipstick ürünün gerçek dünyadaki boyutları metre ile ölçülmüş ve yüksek doğrulukta ölçüm yapıldığı kanıtlanmıştır.

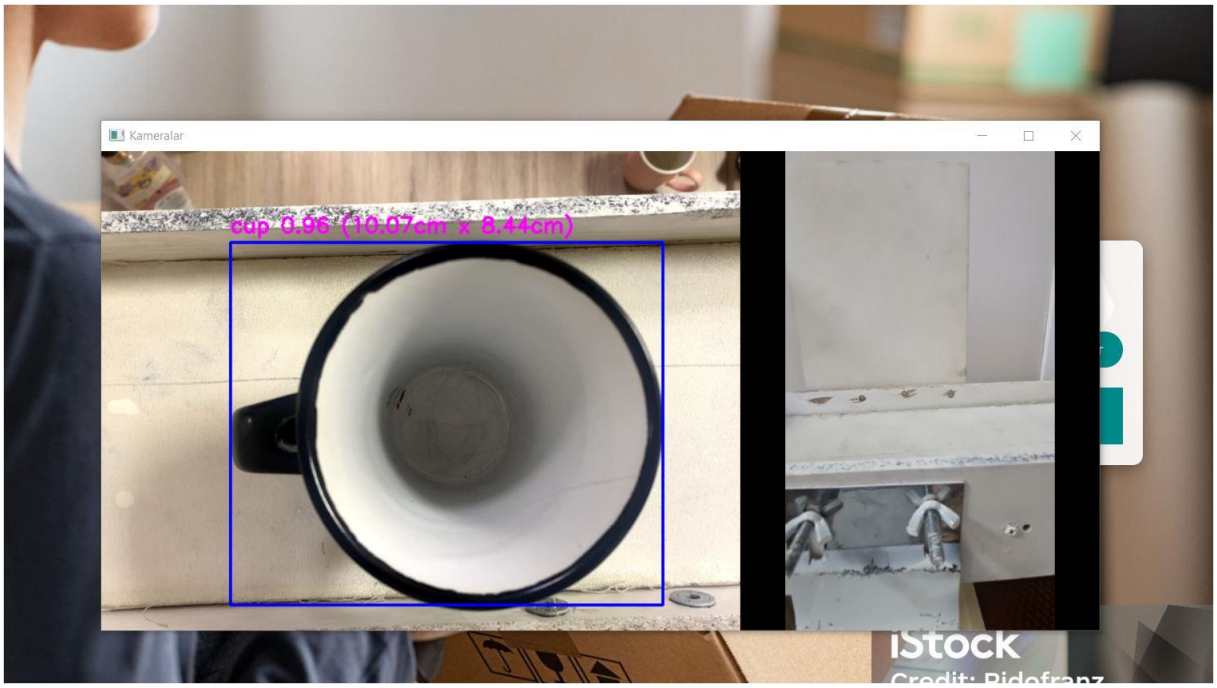


Şekil 20. Yan açı genişlik ölçümü



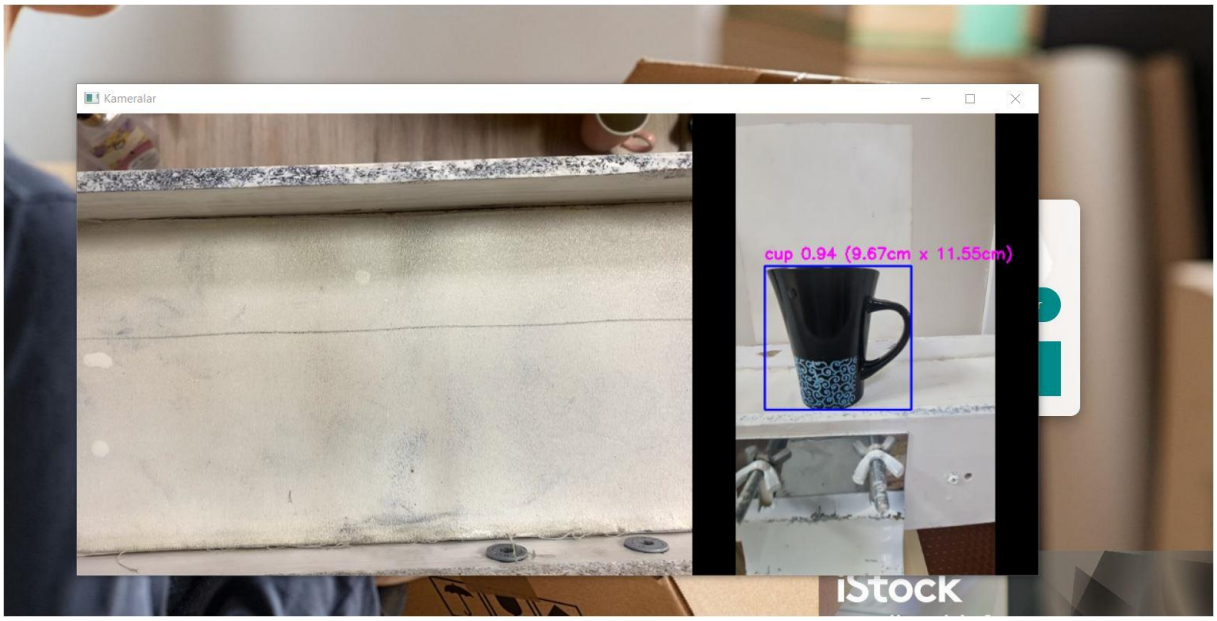
Şekil 22. Yan açı uzunluk ölçümü

Aşağıdaki şekil 22 ve 23'de ise farklı bir sınıfa ait cup nesnesinin tespit edilmesi ve boyutlandırılması gösterilmiştir. Buradan da gösterildiği gibi farklı sınıflardaki nesnelerin tespiti ve boyutlandırılması da yüksek oranda bir skor ile başarıya ulaşmıştır.



Şekil 22. Cup nesnesinin üst açıdan kamera ile tespiti ve boyutlandırılması

Şekiller incelendiğinde başarı oranlarının %94 olduğu görülmektedir. Boyutlandırma konusunda ise çok hassas bir kalibrasyon olduğu ve pixel ile ölçümden geçiş yapıldığı için paketleme için ±2 hata payı bırakılmalıdır.



Şekil 23. Cup nesnesinin yan açıdan kamera ile tespiti ve boyutlandırılması

Projenin ilerleyen aşamalarında, veri setinin farklı nesne sınıfları ve daha çeşitli örneklerle genişletilmesi planlanmaktadır. Bu sayede, modelin farklı türdeki nesneleri tespit etme ve boyutlandırma yeteneği artırılacaktır. Ayrıca, mevcut algoritmaların daha da optimize edilmesi, işlem süresinin kısaltılması ve sistemin genel verimliliğinin artırılması hedeflenmektedir.

Kalibrasyon süreci de projenin önemli bir parçasıdır ve gelecekte bu sürecin iyileştirilmesi üzerinde durulacaktır. Mevcut sistem, referans obje ile kalibrasyon yaparak boyutları belirlemekte oldukça başarılı olsa da, daha hassas ve otomatik kalibrasyon yöntemleri geliştirilerek doğruluk artırılabilir. Örneğin, dinamik kalibrasyon teknikleri kullanılarak, farklı ışık koşulları ve açılardan etkilenmeden doğru ölçümler yapılabilir. Ayrıca, daha gelişmiş kalibrasyon algoritmaları, sistemin farklı ortamlara ve değişken koşullara uyum sağlama yeteneğini artırabilir.

Sonuç olarak, bu proje, nesnelerin doğru bir şekilde tespit edilmesi ve boyutlarının belirlenmesi konusunda başarılı sonuçlar elde etmiş olup, endüstriyel uygulamalar için pratik ve etkili bir çözüm sunmaktadır. Gelecekte yapılacak iyileştirmeler ve optimizasyonlarla, sistemin performansı daha da artırılacak ve daha geniş bir kullanım alanına hitap edebilecektir.

8. PROJE YÖNETİMİ

Bu kısımda projeyi geliştirirken uygulanan adımlar sırasıyla verilmiştir.

8.1. İş-Zaman Çizelgesi

Proje yapılma aşamasında izlenecek adımlar aşağıdaki tablo 2'de verilmiştir.

Tablo 2. Proje Aşamaları

1. Adım	Proje tanımlanır ve literatür araştırması yapılır.
2. Adım	Projede kullanılacak metot, kaynak ve materyaller belirlenir.
3. Adım	Veri seti için veriler toplanır (fotoğraf çekilir.)
4. Adım	Oluşturulan veri seti Google Colabda YoloV8 ile eğitilir
5. Adım	Elde edilen model görüntü işleme algoritmasında kullanılır.
6. Adım	Görüntü işleme algoritmasını yazmak için Python programlama dili ve OpenCV kütüphanesi kullanılır
7. Adım	Nesne tespiti için YOLO derin öğrenme algoritması kullanılır.
8. Adım	Yazılan algoritma arayüze dahil edilmek için django frameworkü ile entegre edilir.
9. Adım	Arayüz Html, Css, Django ile yazılır.
10.	Konveyör bant tasarlanır ve yapılır. Kameralar sabitlenir.
11. Adım	Kameradan gerçek zamanlı olarak gelen görüntüler yakalanır ve nesnelerin boyutları elde edilir.
12. Adım	Elde edilen nesne tipleri ve boyutları veri tabanına kaydedilir.
13. Adım	Veri tabanına kaydedilen veriler arayüzde görüntülenir.

8.2. Risk Yönetimi

Projede belirlenen riskler ve risk planları aşağıdaki tablo 3'de verilmiştir.

Tablo 3. Risk Yönetimi

İP No	En Önemli Riskler	Risk Yönetimi (B Planı)
1	Görüntü füzyonu	Hatalar kamera kalibrasyonundan kaynaklanıyorsa daha hassas bir kalibrasyon yapılabilir. Başka bir sebepten kaynaklı ise farklı algoritmalar denenerek parametre ayarları yapılacaktır.
2	Referans objenin yeterince iyi tanımlanmaması	Başlangıçta referans obje olarak madeni para kullanılması öngörülmektedir. Bu referans objenin modelin başarısında herhangi bir olumsuzluğa yol açtığı görülürse çeşitli diğer objeler kullanılacaktır.
3	Model eğitim parametrelerinin ayarlanamaması	Modelin başlangıç ağırlıkları ve hiperparametreleri dikkatlice ayarlanacaktır. Hiperparametre optimizasyonu için çapraz doğrulama kullanılacaktır.
4	Veri etiketleme hataları	Eğitim verilerinin yanlış etiketlenmesi veya eksik etiketlenmesi, modelin doğruluğunu olumsuz etkileyebilir. Verilerin dikkatlice denetlenecektir ve sürekli kontrol edilecektir.

KAYNAKÇA

[1] Uzun Y., Boyacıgil M.F., (2023). Raspberry PI Tabanlı Görüntü İşleme Uygulaması Geliştirilmesi. Fen Bilimleri Enstitüsü Dergisi 6(2): 1067-1078.

[2] Yılmaz M.A., Sungur C., (2021). Görüntü İşleme ve Robot Kol Tabanlı Çikolata Toplama ve Paketleme Sistemi. Avrupa Bilim ve Teknoloji Dergisi Özel Sayı 30, S. 79-82,

[3] Yağmur D. & Atalı G. (2021). HSL Renk Uzayında Görüntü İşleme ve Morfolojik İşlemler Kullanarak Gerçek Zamanlı Nesne Tespiti ve Sınıflandırması. *Avrupa Bilim ve Teknoloji Dergisi*, (28), 607-613.

[4] Peker M. (2009). Görüntü İşleme Tekniği Kullanılarak Gerçek Zamanlı Hareketli Görüntü Tanıma. Sakarya Üniversitesi Fen Bilimleri Enstitüsü, Yüksek Lisans Tezi.

[5] Türkoğlu İ. (2018). Derin Öğrenme Modelleri ve Uygulama Alanlarına İlişkin Bir Derleme. DÜMF Mühendislik Dergisi 10:2, 409-445

[6] Tan F.G., Yüksel A.S., Aydemir E., Ersoy M., (2021). Derin Öğrenme Teknikleri ile Nesne Tespiti ve Takibi Üzerine Bir İnceleme. Avrupa Bilim ve Teknoloji Dergisi Sayı 25, S. 159-171,

[7] Küçük D., Arıcı N., (2018). Doğal Dil İşlemede Derin Öğrenme Uygulamaları Üzerine Bir Literatür Çalışması

[8] Solak S., Altınışık U., (2017). Görüntü işleme teknikleri ve kümeleme yöntemleri kullanılarak fındık meyvesinin tespit ve sınıflandırılması. Sakarya Üniversitesi Fen Bilimleri Enstitüsü Dergisi, 22 (1), 56-65,

[9] Daş R., Polat B., Tuna G., Derin Öğrenme ile Resim ve Videolarda Nesnelerin Tanınması

ve Takibi. Fırat Üniversitesi Müh. Bil. Dergisi 31(2), 571-581,

[10]. Faisal M. M., Mohammed M. S., Abduljabar A. M., Abdulhussain S. H., Mahmmod B. M., Khan W., (2021). Object Detection and Distance Measurement using AI. 14th International Conference on Developments in eSystems Engineering (DeSE).

[11]. Mahesh M., Reddy V., Reddy A., Reddy C., (2022). Object Detection And Dimensioning Using Opencv. International Journal of Creative Research Thoughts (IJCRT). Volume 10, Issue 6.

[12]. Mogre N., Bhagat S., Bhoyar K., Hadke H., Ingole P., (2022). Real Tıme Object Detectıon And Heıght Measurement. International Research Journal of Modernization in Engineering Technology and Science. Volume 04, Issue 05.

[13]. Jiwane S., Panda A., Basutkar Y., Golande A., (2022). Real-Time Object Measurement Using Image Processing. Mathematical Statistician and Engineering Applications. Volume 71, Issue 4. 3260-3267.

[14] Melenli S., Topkaya A., (2020). Real-Time Maintaining of Social Distance in Covid19 Environment using Image Processing and Big Data.